EEE/EIE Y2 Smart Grid Project

Group: Watt's Up

Chun Wen Ooi (02586872)
Dzuldiniy Hussain Bin Dzulkeflee (02574233)
Fangnan Tan (02571173)
Hong Zhe Tan (02562503)
Wei Zen Ooi (02614796)
Yi Da Wu (02567507)
Yong Zhi Tong (02581094)

Imperial College London

June 2026

Abstract

The Smart Grid is a DC microgrid built from five cooperating SMPS modules. It balances generation, storage and load on a shared 10V bus while minimising net grid cost through import, export and supercapacitor charge or discharge. Each converter regulates its own current flow against a local bus measurement, with a central coordinator setting dispatch commands. The system operates from a 12V primary supply onto a 10V bus with a ±0.1V droop band, allowing each module to act autonomously on the bus voltage signal. A Pico W on every module runs the control loop and feeds INA219 or SPI ADC readings to an operator dashboard. The dispatch algorithm schedules deferrable loads and supercapacitor cycles around price forecasts, beating the naive baseline by 20% in lab runs and by a mean of 28% across 1000 simulated days. A chassis printed in 3D houses the SMPS boards for airflow and mechanical protection.

Word count: ***11946***

Contents

1 Project Objectives & Management 1
1.1 Project Objectives 1
1.2 Project Management System & Time Planning 1
1.3 Role Allocation 3
2 System Architecture & Key Design Decisions 4
2.1 Module Roles 4
2.2 Overall Microgrid Topology 5
2.3 Voltage Architecture: 10V Bus, ±0.1V Droop Band and 12V PSU 5
2.4 Coordination Scheme: Symmetric Droop vs Mutex Arbitration 7
3 PV SMPS 9
3.1 Design & Simulation 9
3.1.1 Topology Selection 9
3.1.2 MPPT Strategy: Fixed Voltage vs Lookup Table vs P&O 9
3.2 Testing & Implementation 10
3.2.1 Bench Noise Treatment 10
3.2.2 P&O Implementation 10
3.3 Evaluation & Optimisation 11
3.3.1 MPPT Mode Comparison 11
3.3.2 Measured SMPS Efficiency 12
4 Supercapacitor SMPS 15
4.1 Design & Simulation 15
4.1.1 Boost Topology and Boot-with-Empty-Capacitor Rationale 15
4.1.2 Operating Window 10.5 to 17.5 V 15
4.1.3 Command-Driven CC: Rationale for the Departure 16
4.2 Testing & Optimisation 16
4.2.1 Automated Cycle-Test Method 16
4.2.2 Round-Trip Efficiency Results 17
4.2.3 Self-Discharge and Hold-Duration Dependence 19
5 Import & Export SMPS 21
5.1 Design & Simulation 21
5.1.1 Topology Choice 21
5.1.2 Droop Setpoints: 9.9 V Import and 10.1 V Export 21
5.2 Testing & Implementation 22
5.2.1 Anti-Windup and Soft-Start Grace 22
5.3 Evaluation & Optimisation 22
6 LED Load SMPS 24
6.1 Design & Simulation 24
6.1.1 Topology 24
6.1.2 Control Mode: CC vs Commanded Power 24
6.2 Testing & Implementation 24
6.3 Evaluation & Optimisation 25
6.3.1 Power-Following Accuracy 25
6.3.2 Bus-Side Measurement Gap 26
7 Measurement Filtering 27
7.1 Hardware Averaging 27

7.2 Exponential Moving Average 27
7.3 Three-Second Block Average 28
7.4 Evaluation 28
8 Software System 30
8.1 Architecture 30
8.2 Backend and Control Layer 31
8.3 Frontend Dashboard 32
8.4 Design Trade-offs 32
8.5 On-Module Fallback Dashboard 33
9 Algorithm Software 34
9.1 Coordinator Loop and Economic Objective 34
9.2 Algorithm Structure 34
9.3 Naive Baseline 35
9.4 Smart V1: Forecast-Aware Scheduling and Arbitrage 36
9.5 Evaluation 38
9.6 Backtest on 1000 Simulated Days 39
9.7 Auxiliary Power Overhead 40
10 Mechanical Design 41
10.1 3D-Modelled SMPS Enclosure 41
11 Extra Test & Requirement Verification 42
11.1 Communication & Integration Tests 42
11.2 Requirements Verification 43
References 44

1 Project Objectives & Management

1.1 Project Objectives

The project delivers a five-module DC microgrid that regulates a shared 10V bus across PV generation, supercapacitor storage, grid import, grid export and a programmable LED load. The control system dispatches power between modules to minimise net cost under a time-varying tariff. The prototype is evaluated on demo day, 18 June 2026, against a published price-and-load scenario.

1.2 Project Management System & Time Planning

Coordinating a seven-member hardware and software team across a five-week delivery window required tooling that off-the-shelf platforms could not provide as a single integrated system. Rather than stitching together Notion, Trello, Microsoft Teams and ChatGPT, each of which solves only a fragment of the problem individually, an AI-powered Project Management System (PMS) was coded by our team for the project. Every layer of the frontend, backend, AI pipeline and database schema was purpose built around our needs.

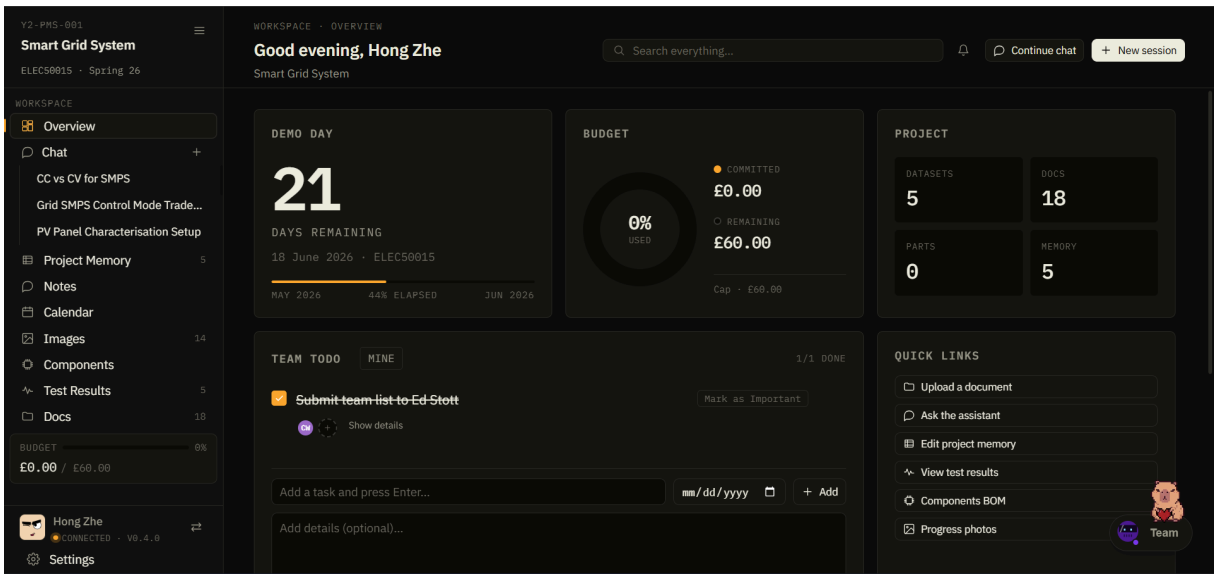


Figure 1.1: PMS dashboard: countdown, budget, stats and todo.

The platform is structured around four modules. A live operational dashboard (Figure 1.1) displays the project's current situation and deadlines. An automated meeting transcript module (Figure 1.2) records, transcribes and structures every meeting into attendee lists, discussion summaries and extracted action items. An AI-driven article and documentation generator named Aria (Figure 1.3) produces project-specific technical writing such as component-selection notes and lab write-ups. A Retrieval-Augmented Generation (RAG) project chat (Figure 1.4) compiles Aria articles, meeting minutes, the GitHub firmware repository, datasheets and test results into context for the AI to analyse and discuss with team members.

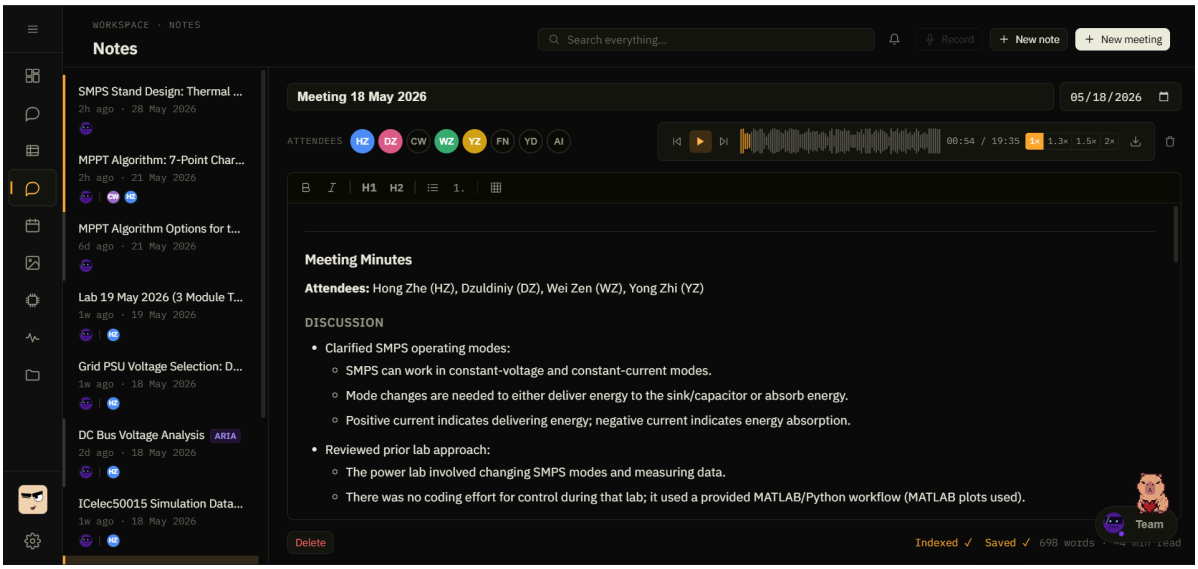


Figure 1.2: Meeting transcript module.

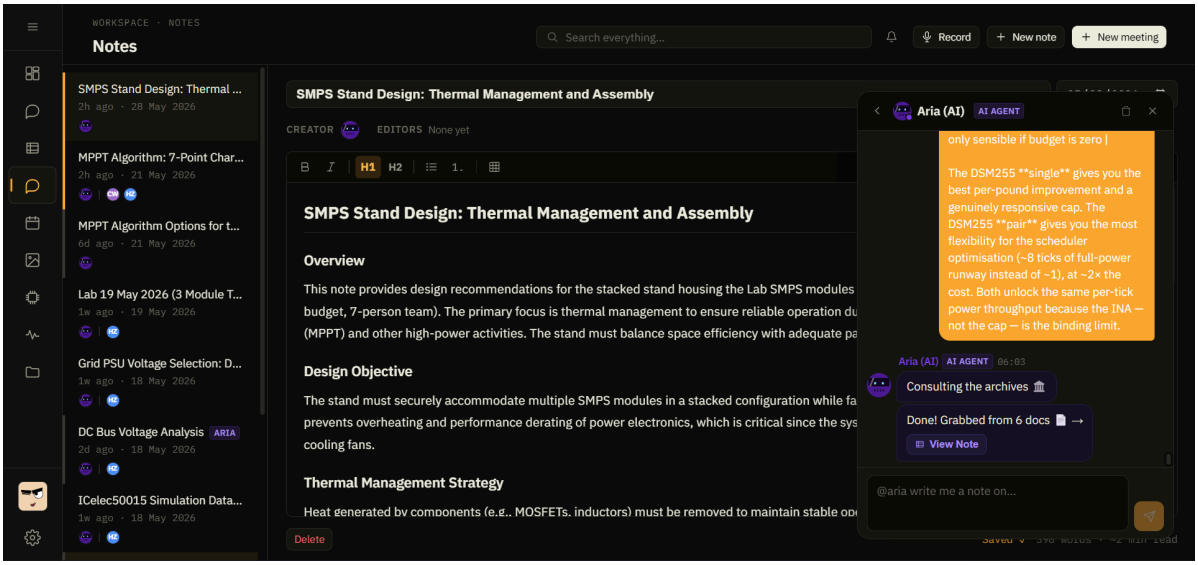


Figure 1.3: Aria article generator.

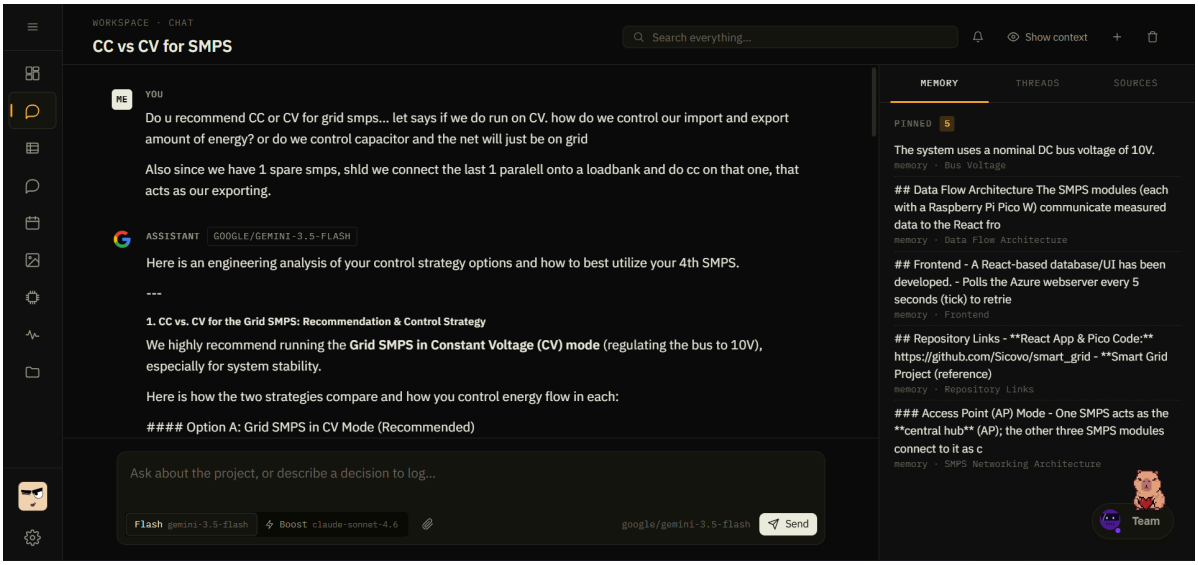


Figure 1.4: RAG project chat with cited sources.

Several additional features were built other than the four core modules (Figure 1.5). A shared calendar with a Gantt-style timeline consolidates every milestone and submission deadline so that everyone is aware of the project timeline. A project gallery centralises lab photographs that would otherwise be unorganised and scattered across personal phones. A test results module captures raw measurement data with built-in charting, removing the need for data transfer between spreadsheet software. Finally, lightweight personalisation through the team buddy selector motivates team members to actively log on to the platform and increases productivity.

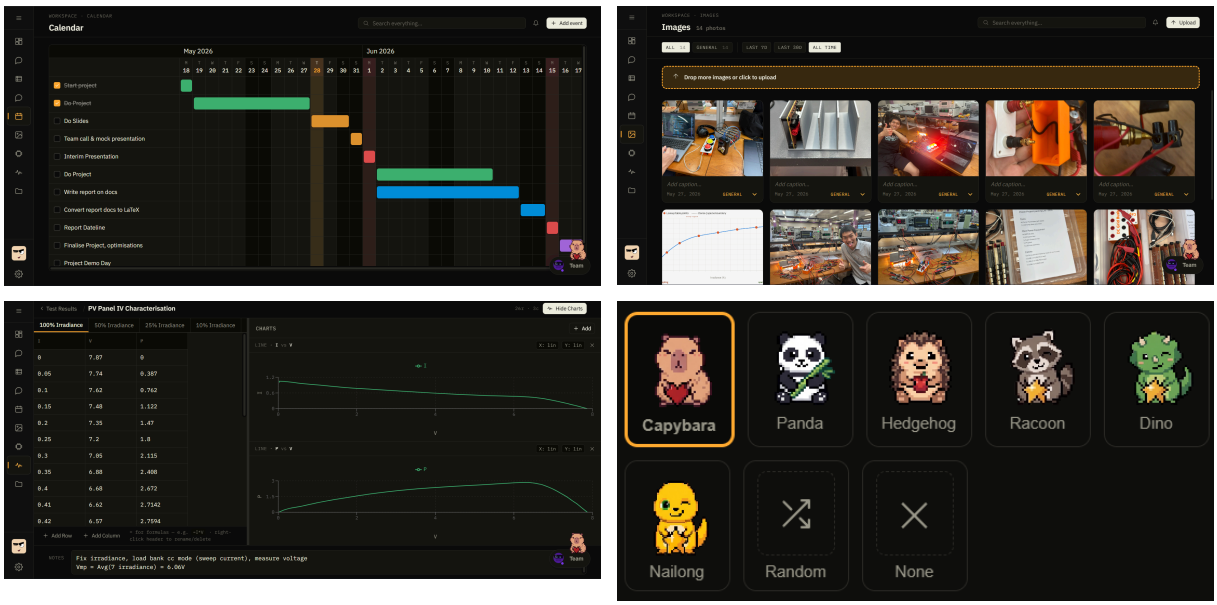


Figure 1.5: Calendar, image gallery, test results, and buddy selector.

1.3 Role Allocation

Work was split into one Task Master and one Task Assistant per hardware module and software stack, summarised in Table 1.1. The Task Master owned the design, build and report writing for their module, while the Task Assistant supported testing, ran cross-checks and reviewed the writing. Report-structure authoring was led by Hong Zhe, with Chun Wen and Yong Zhi acting as report checkers.

	PV Panel	LED Load	Import (PSU)	Export (Load Bank)	Supercapacitor	Backend	Frontend
Task Master	Chun Wen	Yong Zhi	Wei Zen	Yi Da	Hong Zhe	Dzuldiniy	Fangnan
Task Assistant	Hong Zhe	Yi Da	Chun Wen	Wei Zen	Yong Zhi	Fangnan	Dzuldiniy

Table 1.1: Per-module role allocation across hardware and software.

2 System Architecture & Key Design Decisions

This section covers the hardware architecture: module roles, microgrid topology, bus and supply voltages, and the droop coordination scheme. The software architecture, dispatch algorithm and operator dashboard that sit on top are covered separately in Sections 8 and 9.

2.1 Module Roles

Module	External component	Topology	Control	Role on the bus
Grid Import SMPS	12V grid PSU	Buck	CV, droop at 9.9V	Sources current when bus $<$ 9.9V
Grid Export SMPS	Sink resistor	Buck	CV, droop at 10.1V	Sinks current when bus $>$ 10.1V
PV SMPS	PV panel	Boost	MPPT	Harvests panel power into bus
Supercapacitor SMPS	10.5 to 17.5V supercapacitor	Bidirectional buck/boost	CC, commanded	Stores or releases on command
LED Load SMPS	3V LED module	Buck (internal)	CC	Drives the LED load

Table 2.1: Per-module topology, external component, control mode and bus role.

The system comprises five SMPS modules connected to a shared 10V DC bus. The import and export modules share a synchronous-buck topology and a common droop, differing only in setpoints; this creates the dead-band discussed in Section 2.4. The Supercapacitor SMPS does not droop and runs as a commanded current source, so the economic algorithm in Section 9 can use it to buy energy cheap and sell it expensive. The LED Load SMPS is internally managed; only its bus-side draw is exposed. All four bidirectional modules use the same power stage in Figure 2.1, with a 100 µH inductor and a 100 mΩ shunt on Port B. The roles in the table depend on the port connections, not on the hardware, so per-module sections refer back to this single stage.

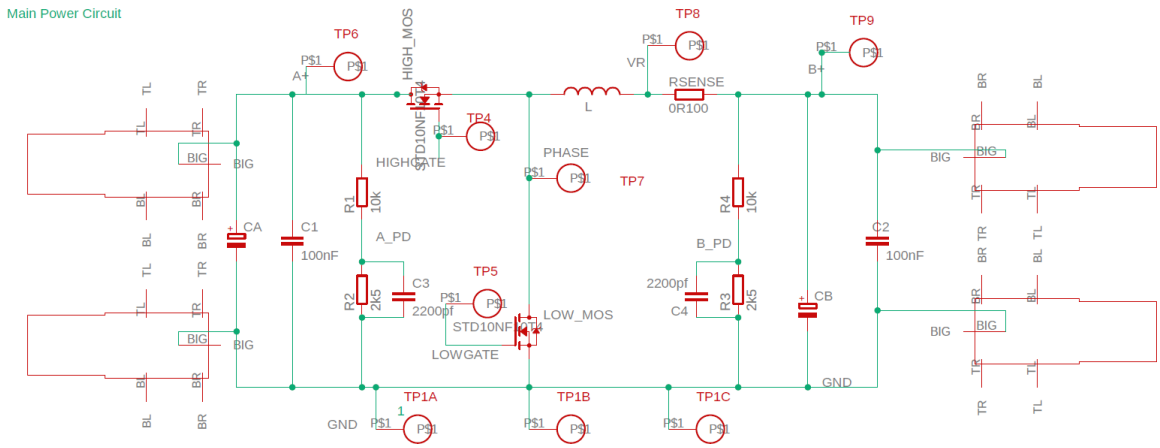


Figure 2.1: Bidirectional SMPS power stage, common to all four modules [1].

2.2 Overall Microgrid Topology

Figure 2.2 shows the complete system. External components sit at the top, each connected through its SMPS module to the shared 10V DC bus. A dedicated Pico W on every module runs the local control loop. A FastAPI backend checks the Azure-hosted web service [2] once per second for a new simulation tick, fetching the demand, price, irradiance and deferrable data when one appears; each new tick triggers the dispatch decision (Section 9), whose setpoints are pushed to each module over HTTP. The operator dashboard renders the live readings from the same backend.

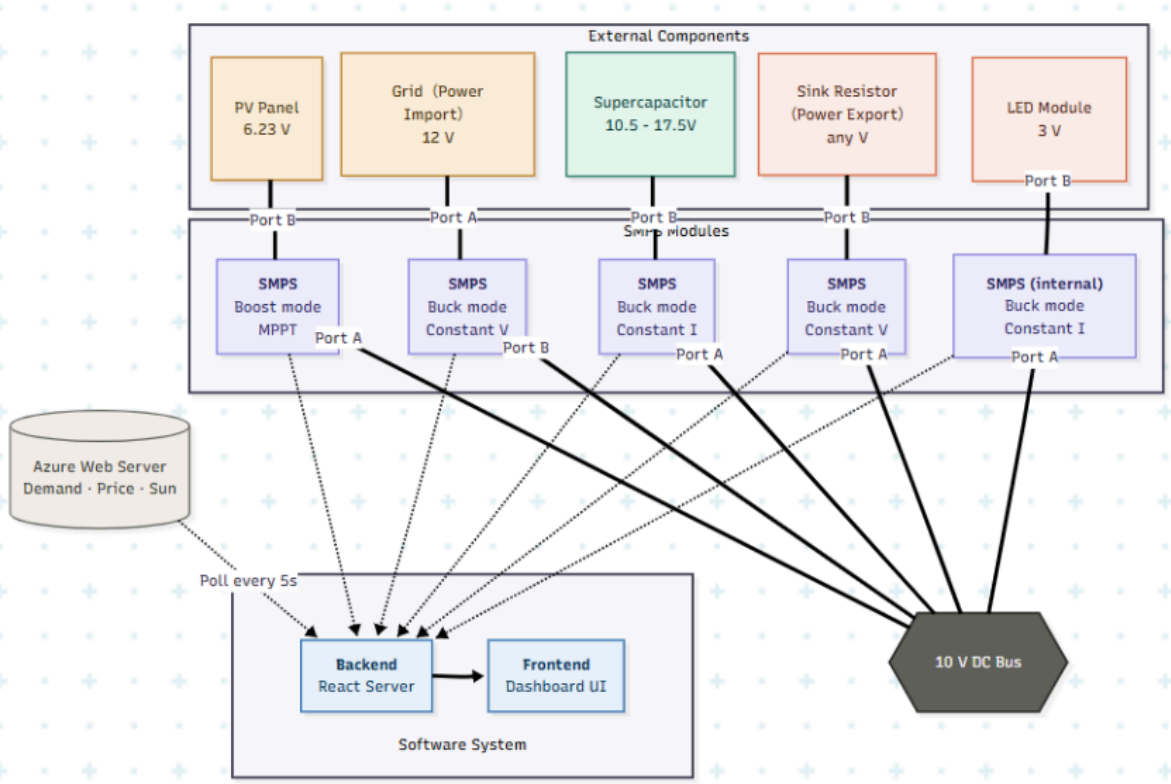


Figure 2.2: Smart Grid system architecture and power flow.

A few properties of this topology come up again in later sections. Every module is autonomous in its bus-regulation role, acting on its local V_{bus} measurement alone while the backend supplies only setpoints and economic context. All demo-day power measurements occur at the bus, never at SMPS terminals. And the supercapacitor is the only element that stores energy for longer than the bus capacitance can, which puts its commanded behaviour at the centre of the economic optimisation.

2.3 Voltage Architecture: 10V Bus, ±0.1V Droop Band and 12V PSU

The voltage architecture is fixed by the operating windows of the PV panel, supercapacitor and grid-interface converters. A 10V nominal bus is the smallest round-number voltage that sits below the supercapacitor charge window, above the PV panel open-circuit voltage of about 8.5V, and reachable from a 12V buck input (justified below). Raising the bus would add switching stress with little benefit; lowering it would reduce the supercapacitor margin and increase the PV boost ratio.

Each droop-controlled module follows a local linear control law of the form

$$I_{\text{module}} = \frac{V_{\text{set}} - V_{\text{bus}}}{R_{\text{droop}}}$$

with sign and saturation set by module role. R_{droop} is sized so each module reaches its rated current at the edge of the ± 0.1V band, bounding the full-load steady-state bus error to $\pm 1\%$ of nominal. This keeps the bus stiff enough for downstream modules while regulation stays local.

The bus voltage choice also affects the scoring of converter losses. Demo-day measurements are taken at the bus, so LED Load and Export losses occur after the measured point and are counted as load or dissipated export power; their efficiency does not affect the score directly. Import losses occur before energy reaches the bus, so every watt lost between the 12V supply and the bus is grid energy missing from the measured balance.

The 12V PSU is the lowest practical input that preserves buck regulation with sufficient headroom, while avoiding the switching loss a higher input would add. For a synchronous buck converter [3],

$$P_{\text{cond}} = D \cdot I_{\text{rms}}^2 \cdot R_{\text{ds,on,HS}} + (1 - D) \cdot I_{\text{rms}}^2 \cdot R_{\text{ds,on,LS}}$$

which reduces to

$$P_{\text{cond}} = I_{\text{rms}}^2 \cdot R_{\text{ds,on}}$$

for matched MOSFETs. The dominant conduction term is therefore independent of both D and V_{in}. Switching loss scales with V_{in} or V_{in}^2:

$$P_{\text{sw}} \approx \frac{1}{2} \cdot V_{\text{in}} \cdot I_{\text{load}} \cdot \left(t_r + t_f\right) \cdot f_{\text{sw}} + \frac{1}{2} \cdot C_{\text{oss}} \cdot V_{\text{in}}^2 \cdot f_{\text{sw}}$$

and inductor ripple shrinks as V_{in} approaches V_{bus}:

$$\Delta I_L = (V_{\text{in}} - V_{\text{bus}}) \cdot D \cdot \frac{T}{L}$$

The binding constraint is dropout, not efficiency: with $D = \frac{V_{\text{bus}}}{V_{\text{in}}}$, duty tends to unity as the input approaches V_{bus}, but a real converter cannot reach $D = 1$ due to minimum off-time, gate-driver refresh and maximum-duty limits. With a maximum duty of about 0.9, regulation is lost below $\frac{V_{\text{bus}}}{0.9} \approx 11$V. A 12V bench supply leaves margin above this floor while staying close enough to the bus to limit switching loss.

Figure 2.3 shows a flat usable region rather than a strong efficiency optimum: across 11 to 15V at a 10V bus and 1.5A load, modelled loss changes by only tens of milliwatts, about 0.2% of delivered power. The sharp collapse below this region is dropout. The 12V point sits on the flat plateau with margin to spare; a future confirmation sweep from 11 to 14V at $V_{\text{bus}} = 10$V would verify the predicted shallow efficiency rise and the dropout cliff.

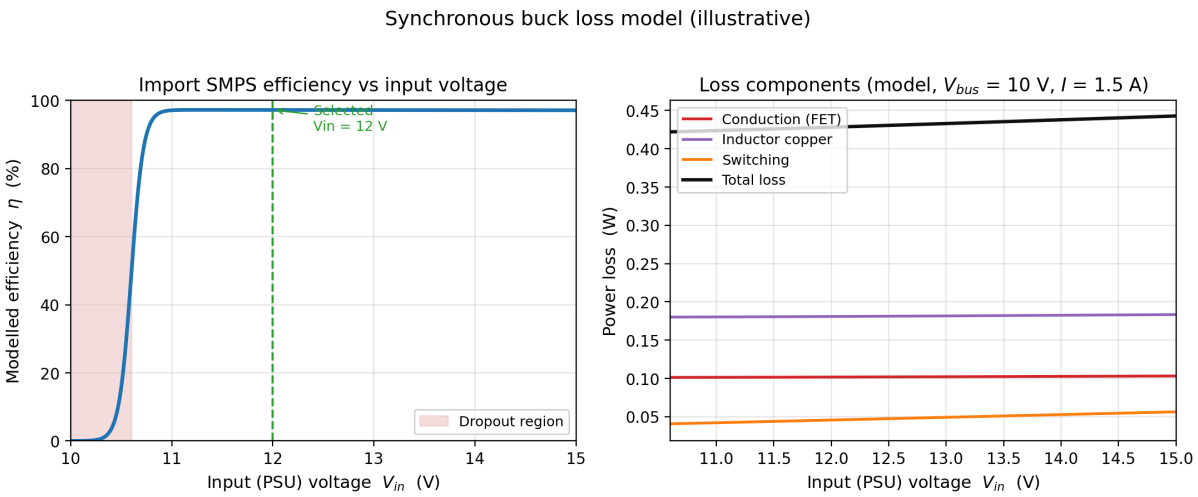


Figure 2.3: Modelled buck efficiency and loss split versus input voltage.

2.4 Coordination Scheme: Symmetric Droop vs Mutex Arbitration

Two architectures were considered. Mutex arbitration assigns bus control to one module at a time, with a coordinator switching modes as conditions change; symmetric droop, the established primary-control approach for DC microgrids [4], lets every module act at once on its local V_{bus} measurement. Mutex gives tighter steady-state regulation, but it places the communication channel inside the safety-critical bus-regulation loop (a dropped link or stalled coordinator collapses the bus), and every handover adds a voltage transient. Droop keeps each loop local and continuous: a communications failure degrades the economic optimisation but not bus stability, the curves blend without handover transients, and the cost is a small steady-state error bounded to ± 0.1V by design (Section 2.3).

Symmetry is enforced by mirroring the import and export setpoints about the 10V nominal:

$$V_{\text{set,import}} = 9.9 \text{ V} \qquad V_{\text{set,export}} = 10.1 \text{ V}$$

This creates a dead-band $V_{\text{bus}} \in [9.9, 10.1]$V in which neither import and export module acts and the bus is held by the PV module and supercapacitor alone, the no-cost configuration the algorithm in Section 9 maximises. Outside it, the system falls back to grid import or export with no explicit mode-change logic. Figure 2.4 and Figure 2.5 show the droop curves and a load cycle moving between import, dead-band and export regimes without central arbitration.

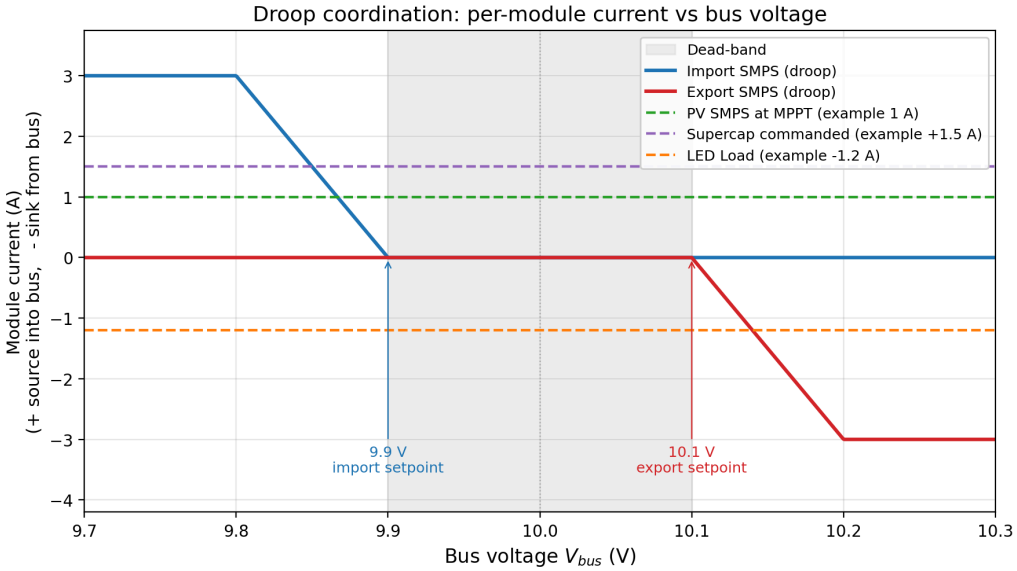


Figure 2.4: Droop curves: module current versus bus voltage.

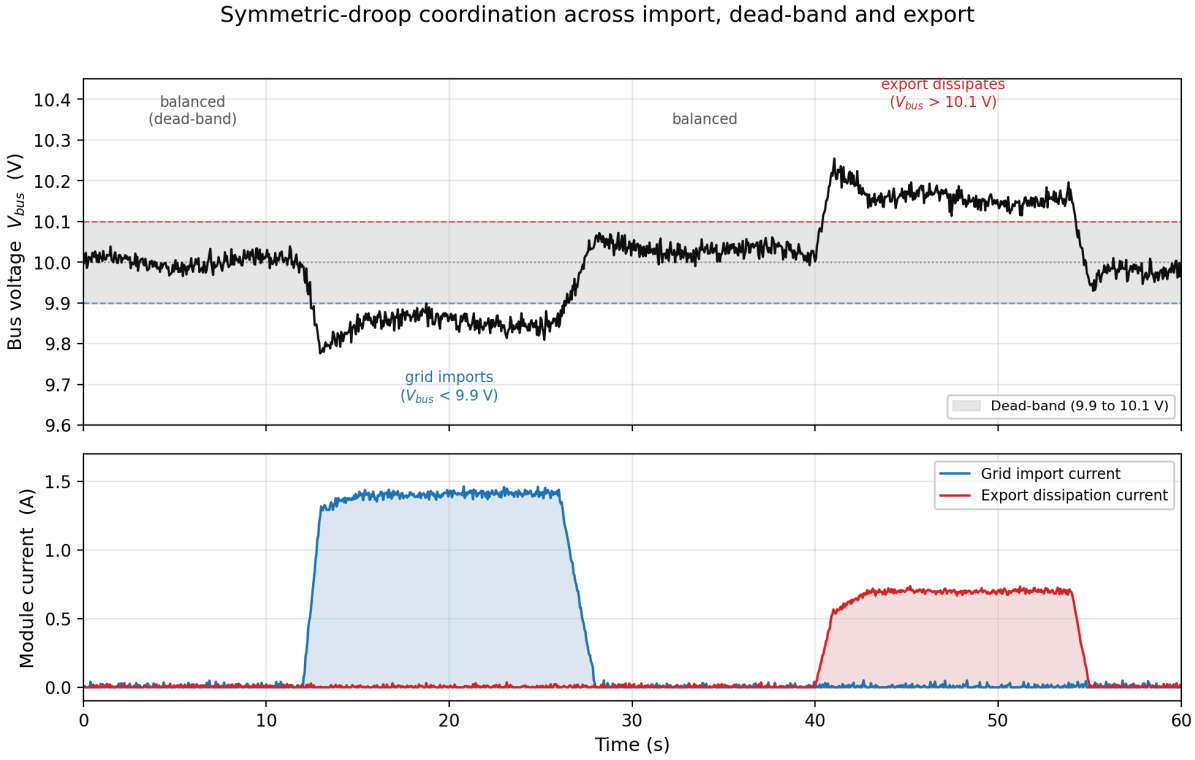


Figure 2.5: Bus voltage and converter currents across import, dead-band and export.

3 PV SMPS

3.1 Design & Simulation

3.1.1 Topology Selection

The bench setup uses one PV panel rated at approximately 4W under full bench-emulator illumination at room temperature, with a datasheet open-circuit voltage of 8.5V and short-circuit current of 1.0A. Demo day adds a second identical panel in parallel for roughly double the available harvest, sharing the same MPP voltage but doubling the current. The single-panel characterisation in Section 3.3 transfers directly to the demo-day configuration: the per-panel MPP voltage is unchanged, so the MPPT lookup table, the P&O dwell behaviour and the boost converter's operating duty all stay valid; only the per-panel current at the bus doubles, which sits well under the 1.5A inductor current cap. Under the solar emulator the measured open-circuit voltage was about 7.5V, with the MPP near 5.20V at full irradiance and 5.10V at low irradiance (Figure 3.1). Each curve was taken by stepping the panel voltage from 2V toward the open-circuit knee (with denser sampling around the MPP) at one of six emulator dimmer settings (25% to 100%). Since the MPP voltage (around 5V) is below the 10V bus, energy reaches the bus by stepping up, so the module runs as a boost converter.

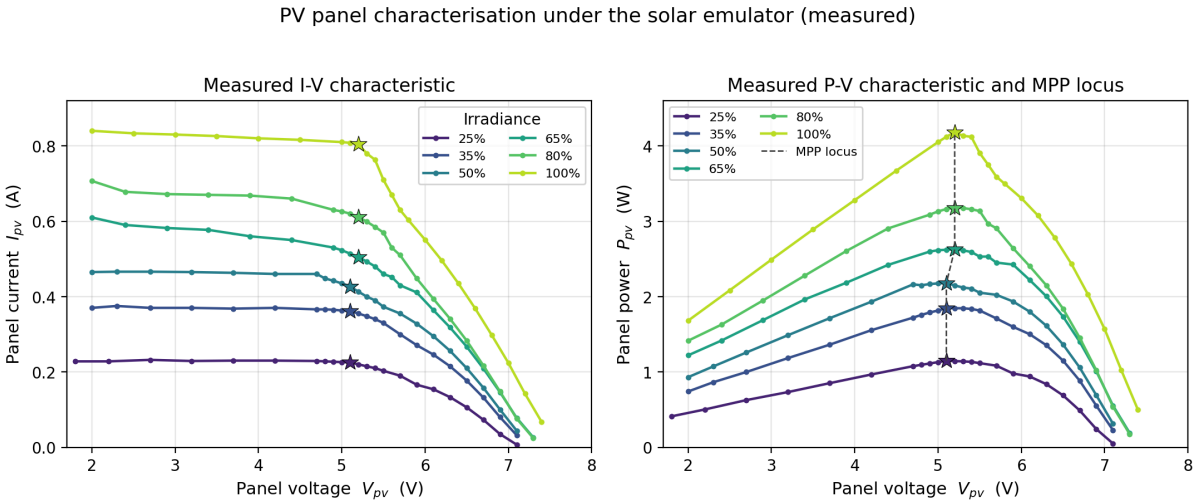


Figure 3.1: Measured PV panel I-V and P-V curves with the MPP points.

The panel is wired to Port B (lower, variable voltage) and the bus to Port A (fixed 10V), satisfying the hard constraint $V_{\mathrm{A}} > V_{\mathrm{B}}$. The BU/BO switch is set to BOOST, and unlike the buck modules the PWM is not inverted: the firmware writes duty directly rather than the `65536 - pwm_out` form.

Current is sensed by the INA219 across the 0.10 Ω Port B shunt. The shunt is wired in the Port A to Port B sense direction, so the firmware negates the reading; a positive inductor current then corresponds to panel-to-bus boost flow. The controller is cascaded: an outer voltage PI regulates the panel voltage to the commanded V_{mpp}, and an inner current PI drives the duty. The operating point is set by whichever MPPT strategy is active.

3.1.2 MPPT Strategy: Fixed Voltage vs Lookup Table vs P&O

Three MPPT approaches were considered and prototyped: a static constant-voltage operating point (Fixed), an irradiance-indexed lookup table (LUT) driven by the Azure `/sun` feed, and

online Perturb and Observe (P&O). Figure 3.2 shows V_{mpp} varying by only 0.10V across the full irradiance range, so a fixed 5.15V setpoint would lose under 1% of available harvest at every characterised point at thermal equilibrium. The trade-offs are summarised in Table 3.1.

Mode	Description	Benefits	Disadvantages
Fixed	Constant V_{mpp} at 5.15V.	Simple; no external feed; no oscillation.	Open-loop on temperature.
LUT	V_{mpp} interpolated from the Azure `/sun` value.	Tracks irradiance; immediate settling.	Needs characterisation and the web link; open-loop on temperature.
P&O	Perturbs V_{mpp}, keeps or reverses by power comparison [5].	No external feed; corrects temperature drift.	Oscillates around the peak; noise-sensitive; slower convergence.

Table 3.1: MPPT mode comparison: Fixed, LUT and P&O.

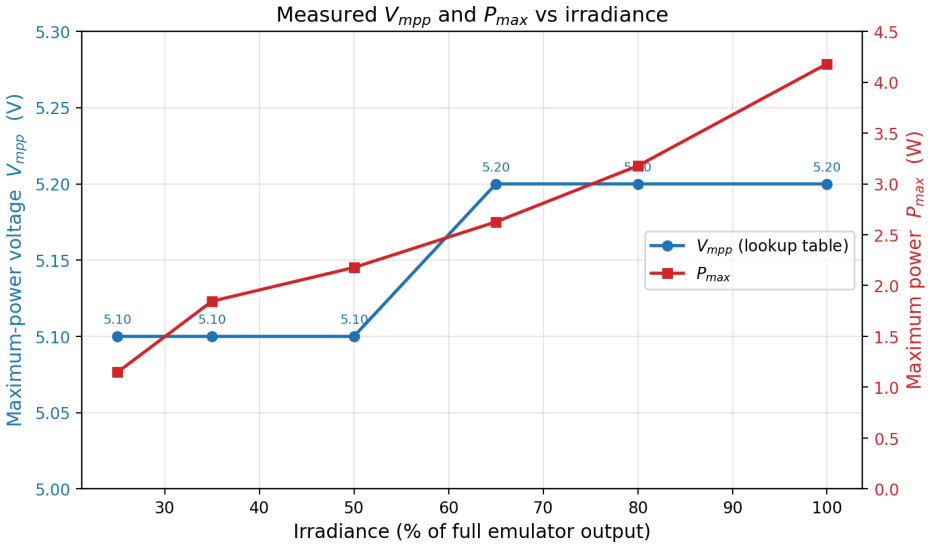


Figure 3.2: Measured Vmpp and Pmax versus irradiance: the MPPT lookup table.

Shipping all three gives a fair comparison under identical conditions and a fallback path if the web link fails: LUT drops to Fixed mode, and P&O stays available as an autonomous fallback.

3.2 Testing & Implementation

3.2.1 Bench Noise Treatment

The three-layer noise treatment (Section 7) was applied to the PV readings before testing began; the filtered values feed the dashboard and the P&O power comparison while raw readings are retained for the control loop and trip guards.

3.2.2 P&O Implementation

P&O uses a two-timescale structure. The outer voltage PI tracks the commanded V_{mpp} at 100 Hz; perturbations occur at a 300 ms interval (T_{dwell}), roughly five times the outer-loop settling time, so the power measured at the end of a dwell reflects steady state, not the PI transient. The comparison power is the filtered $V_{\text{panel,lp}} \cdot I_{\text{L,lp}}$ accumulated over the second half of each dwell and averaged. Two-stage filtering (EMA then dwell average) brings the per-comparison noise floor under 10 mW, well below the $P(V_{\text{mpp}})$ curvature near the peak.

A direction-flip rule with a dead-band stops the three-point oscillation degenerating into noise. If dwell-averaged power dropped by more than $\varepsilon = 20$ mW versus the previous dwell, direction reverses; otherwise it continues. The commanded V_{mpp} is hard-clamped to 4.75V to 5.5V, a narrow window centred on the measured 5.10 to 5.20V MPP range (Figure 3.2), so an algorithm fault cannot push the operating point into open-circuit or short-circuit territory. On entry to P&O mode the state is reseeded from the last commanded fixed-mode V_{mpp}, avoiding a stale value. A three-second block average of P_{panel}, restarting on each mode switch, was added to the firmware data so the operator can read steady-state delivered power for each MPPT mode straight from the dashboard; the live trace captures tracking noise separately.

3.3 Evaluation & Optimisation

This section compares the three MPPT modes, then reports the measured conversion efficiency of the SMPS.

3.3.1 MPPT Mode Comparison

Each MPPT mode is tested at 25%, 50%, 75% and 100% irradiance. At each setting the three-second block-averaged P_{panel} is recorded in fixed mode, LUT mode, and P&O mode. $V_{\text{mpp,target}}$ and v_{panel} are logged throughout so the mode-switch transient can be inspected. The primary metric is steady-state delivered power; the secondary metric is the mode-change transient.

Mean recorded powers per mode and irradiance are summarised in Figure 3.3 and tabulated below.

Irradiance	Fixed (W)	LUT (W)	P&O (W)	Spread (mW)
25%	0.767	0.770	0.770	3
50%	1.933	1.932	1.933	1
75%	2.845	2.841	2.841	4
100%	3.577	3.562	3.550	27

Table 3.2: Recording-phase mean panel power per MPPT mode and irradiance.

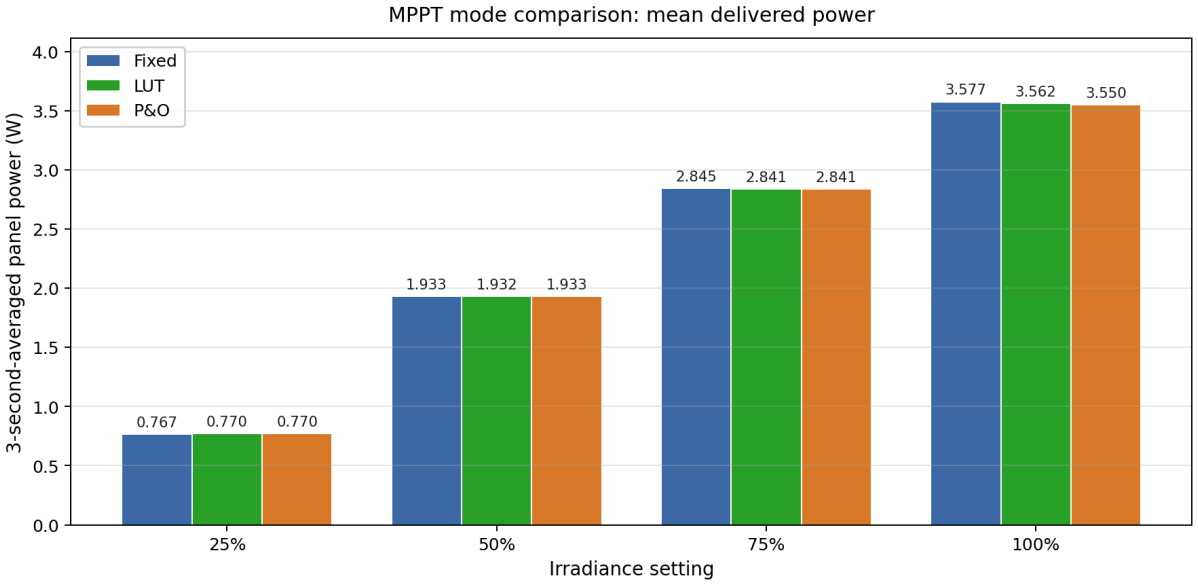


Figure 3.3: Mean delivered panel power by MPPT mode at four irradiance settings.

The three modes converge to within a few mW at every irradiance; the worst-case spread of 27 mW at 100% is under 1% of the 3.55 W operating point. This is what the construction predicts: P&O hunts to the same peak the LUT is built from, and Fixed sits within 0.10V of V_{mpp} across the full range (Figure 3.2), so all three settle close to the same operating point. At thermal equilibrium under the bench emulator there is no steady-state harvest advantage of P&O over LUT or Fixed.

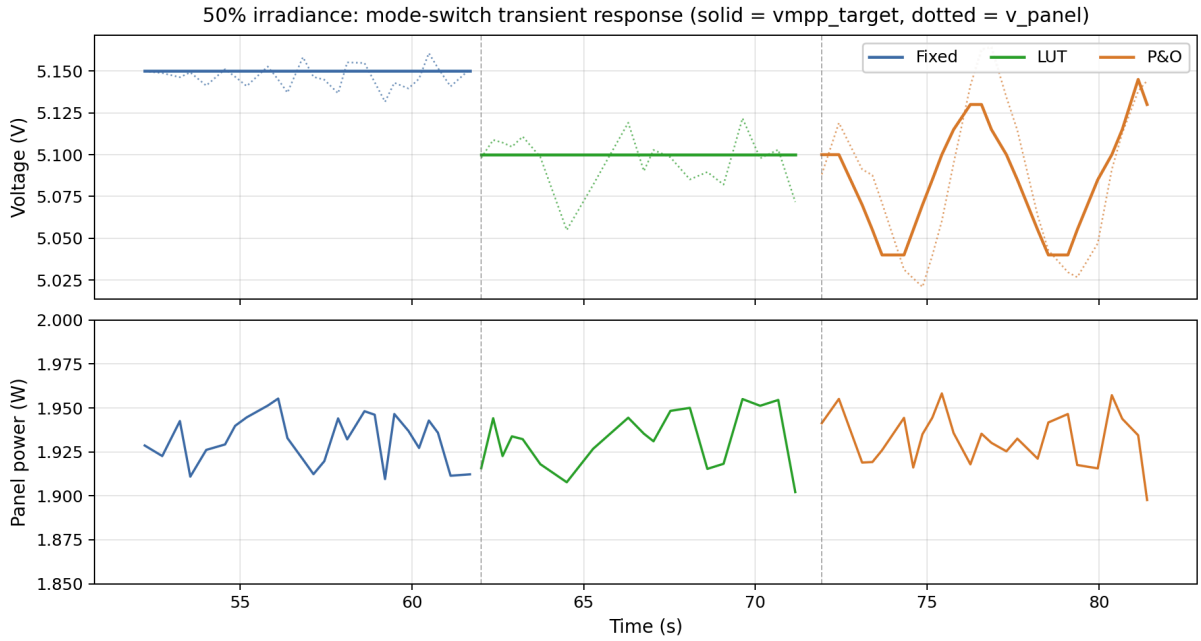


Figure 3.4: Mode-switch transient at 50% irradiance: commanded V_{mpp} (solid), measured v_{panel} (dotted), and resulting P_{panel}.

Figure 3.4 shows the mode-switch transient at 50% irradiance. Fixed and LUT hold their commanded V_{mpp} flat at 5.15V and 5.10V, with v_{panel} tracking inside the noise envelope. P&O shows the expected three-point hunting oscillation around the peak, stepping V_{mpp} up and down in a bounded cycle; the resulting panel power stays inside the same envelope as the other two modes, confirming the algorithm locks onto the peak rather than drifting.

At bench thermal equilibrium all three modes converge to the same peak, so P&O shows no steady-state advantage, but it does confirm the algorithm tracks correctly with its oscillation contained within the noise envelope. The real differentiator is temperature: as the panel warms under sustained illumination, V_{mpp} drifts downward by several mV/°C [6]. Fixed and LUT cannot follow this since both are open-loop, whereas P&O corrects automatically by optimising power directly. P&O is therefore the shipped default, with Fixed and LUT kept as fallbacks in case of a fault.

3.3.2 Measured SMPS Efficiency

PV conversion efficiency is defined as $\eta_{\text{PV}} = \frac{P_{\text{bus,out}}}{P_{\text{panel,in}}}$, with $P_{\text{panel,in}} = V_{\text{panel}} \cdot I_{\text{L}}$ from firmware (both factors are Port B) and $P_{\text{bus,out}} = V_{\text{bus}} \cdot I_{\text{bus,out}}$ from a DMM in series with the PV-to-bus wire (Port A is uninstrumented). Figure 3.5 and the table below show the 48-point sweep: four irradiance settings (25%, 50%, 75%, 100%) at twelve bus-voltage points each, with the panel pinned at its MPP ($V_{\text{panel}} \approx 5.15$ V, from Figure 3.2). The boost duty $D = 1 - \frac{V_{\text{panel}}}{V_{\text{bus}}}$ spans 0.28 to 0.59 across the sweep, staying inside the flat boost plateau.

V_{bus} (V)	Duty D	η_{PV} at 25%	η_{PV} at 50%	η_{PV} at 75%	η_{PV} at 100%
7.11	0.28	96.56%	96.69%	97.17%	97.68%
7.61	0.32	96.36%	96.79%	96.82%	97.89%
8.11	0.37	96.62%	96.57%	96.98%	97.74%
8.61	0.40	96.52%	96.44%	96.73%	98.00%
9.12	0.44	96.16%	96.63%	96.79%	97.86%
9.61	0.46	96.69%	96.34%	97.11%	98.10%
10.21	0.50	96.56%	96.80%	97.15%	98.02%
10.63	0.52	96.57%	96.68%	97.28%	97.79%
11.07	0.54	96.18%	96.75%	96.90%	98.10%
11.63	0.56	96.37%	96.51%	96.92%	98.08%
12.06	0.57	96.32%	96.88%	96.98%	97.83%
12.55	0.59	96.66%	96.84%	96.81%	97.77%

Table 3.3: Measured PV SMPS efficiency across bus voltage and irradiance.

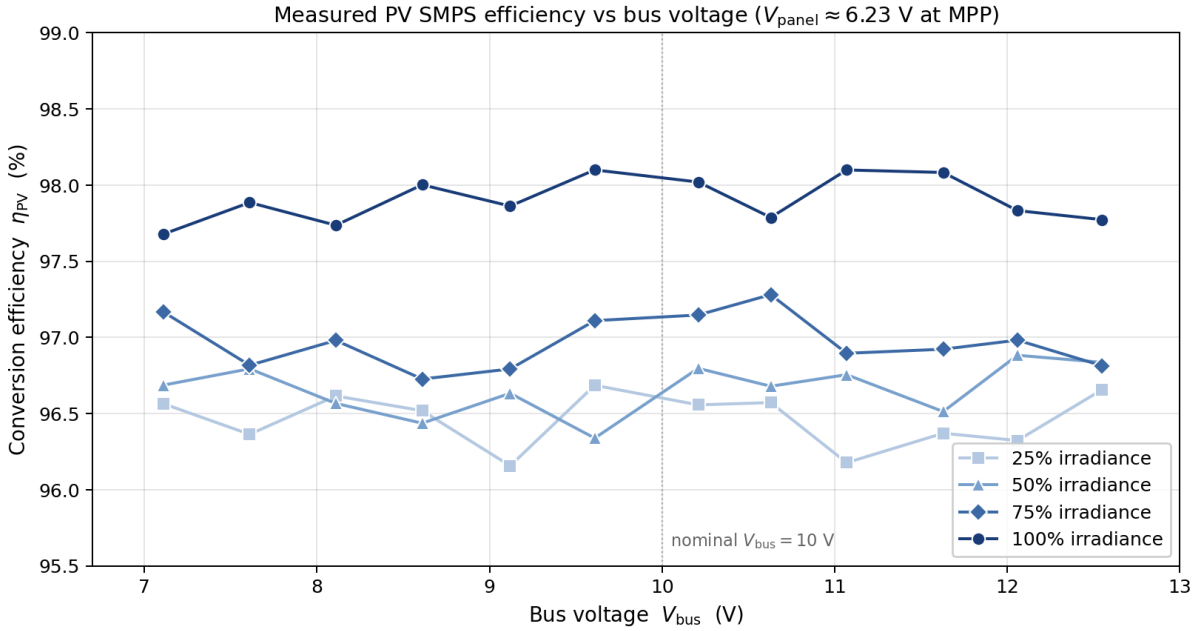


Figure 3.5: Measured PV SMPS efficiency versus bus voltage at four irradiances.

The most obvious feature of the sweep is how flat each η_{PV} trace is with bus voltage. The four curves vary by no more than $\pm 0.3\%$ across their twelve points, with no monotonic trend against V_{bus}, matching the flat plateau of the boost η-versus-D characteristic over the $D = 0.28$ to 0.59 range. Fig. 5 of [7], reproduced in Figure 3.6, shows this plateau across moderate duty cycles and a sharp CCM cliff beyond $D \approx 0.7$. At the supercapacitor ceiling $V_{\text{bus}} = 17.5$V the reachable $D_{\max} = 1 - \frac{5.15}{17.5} \approx 0.71$ sits at the cliff threshold, so the supercapacitor-full corner is the one regime where degraded efficiency would be expected; elsewhere the plateau represents converter behaviour across its operating window.

The other clear trend is that the per-irradiance means rise with irradiance: 96.5% at 25%, 96.7% at 50%, 97.0% at 75% and 97.9% at 100%. The 1.4 percentage-point spread matches the

boost loss model. Fixed gate-drive and switching overhead are independent of throughput, so they consume a larger fraction at low panel power: 100% irradiance gives about 2.3W out, 25% gives about 0.8W, and the same fixed overhead is a bigger share of the lower throughput. The irradiance-to-irradiance gap sits well above the per-point scatter, so the ordering is real rather than a noise effect.

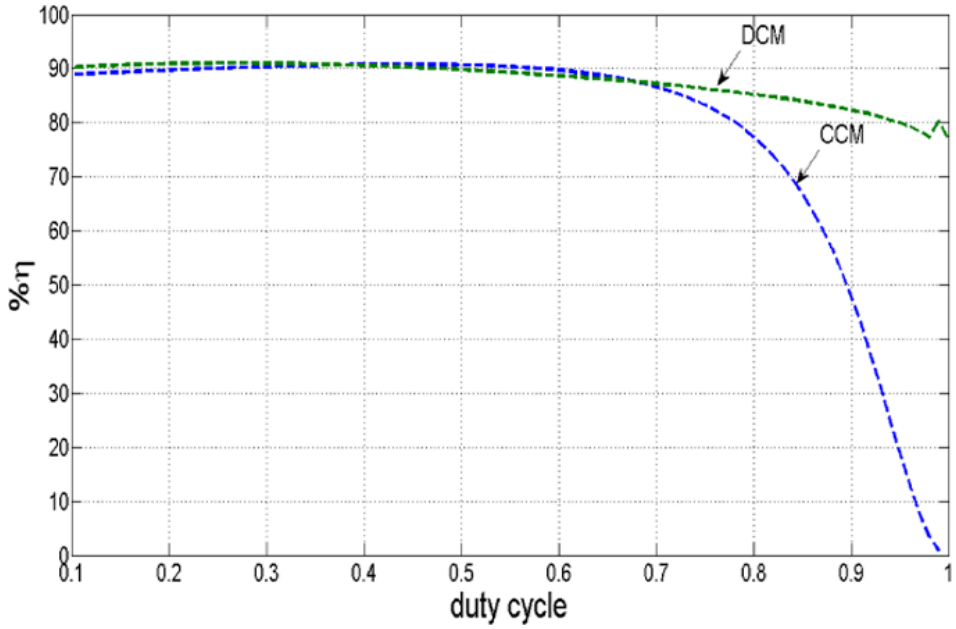


Figure 3.6: Boost efficiency versus duty cycle in DCM and CCM, from Fig. 5 of [7].

PV conversion efficiency stays within 96 to 98% across all four irradiance levels, rising 1.4 percentage points from 25% to 100% irradiance with no meaningful dependence on bus voltage. Import efficiency does enter the score under the dual-meter scoring of Section 2.3 and rises steadily with V_{bus} (Section 5.3), so the import lens marginally favours a higher bus voltage; the PV lens is neutral. The $V_{\text{bus}} = 10$V choice rests on hardware constraints (supercapacitor lower bound, PV panel open-circuit voltage, buck dropout), with both bus-relevant efficiency curves consistent with that pick.

4 Supercapacitor SMPS

4.1 Design & Simulation

4.1.1 Boost Topology and Boot-with-Empty-Capacitor Rationale

The supercapacitor is the only variable-voltage element on the storage side, ranging from 10.5V when depleted to 17.5V when full. The same Port A above Port B transfer rule that fixed the PV topology dictates the port assignment here (the shared power stage in Figure 2.1): the capacitor is the higher and variable rail and must sit on Port A, with the fixed 10V bus on Port B. Energy flows from bus to capacitor (B to A) when charging and from capacitor to bus (A to B) when discharging, so the module is a bidirectional SMPS.

The SMPS is set to BOOST mode, the single most important decision for robustness. In BOOST mode, the on-board circuitry draws from Port B, the bus, which is always energised by the grid and PV modules. As a result the SMPS boots and stays alive even when the capacitor is fully empty at 0V. The alternative BUCK setting would power the bus from Port A, the capacitor, and would trip the SMPS whenever the capacitor fell below the 6V to 7V minimum support voltage. The current sign convention follows the PV module: a positive inductor current means charging, a negative current means discharging, and the dashboard command i_{cmd} is signed accordingly.

4.1.2 Operating Window 10.5 to 17.5 V

The usable voltage window is determined by two hard limits, illustrated in the stored-energy curve in Figure 4.1. The ceiling $V_{\text{cap,max}} = 17.5\text{V}$ is the SMPS port limit and sits under the 18V capacitor rating. The floor $V_{\text{cap,min}} = 10.5\text{V}$ provides a 0.5V margin above the 10V bus to maintain $V_{\text{A}} > V_{\text{B}}$ under ESR-induced voltage drop and measurement noise. Within this window the capacitor stores energy as $E = \frac{1}{2}CV^2$ with $C = 0.5\text{F}$ (two 0.25F capacitors in parallel).

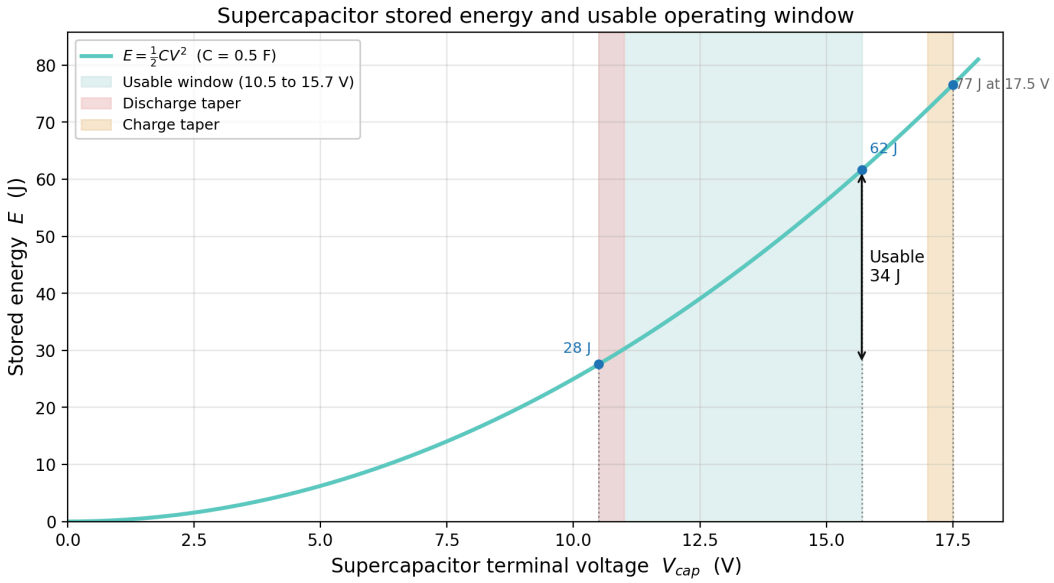


Figure 4.1: Supercapacitor stored energy and usable operating window.

In firmware the practical voltage ceiling is lowered to 15.7V, to reduce the impact of noise overshooting the voltage limit if 17.5V was used instead. The current is reset to 0 whenever the capacitor approaches within 0.5V of the upper and lower limits. The energy genuinely

transferred between 10.5V and 15.7V is approximately 34J of the 49J the full window could hold. A significant complication is the capacitor's approximately 4 Ω equivalent series resistance (ESR): immediately after a current step the voltage ripple is dominated by the resistive drop and charge redistribution, so an accurate voltage can only be determined after some settling time. This directly shapes the cycle-test settle phases described in Section 4.2, which read the open-circuit voltage only after a deliberate pause. Peak current is clamped to 0.60A, leaving approximately 21% headroom under the 0.76A SMPS hardware limit [8].

4.1.3 Command-Driven CC: Rationale for the Departure

Unlike the grid and export modules, the supercapacitor is not involved in droop. The bus voltage is already regulated by the grid and export droop pair (Section 2.4). Instead the capacitor is a commanded current source and sink: it tracks a signed current given by input from the economic algorithm. This is the correct implementation for storage where it allows arbitrage: charging when energy is cheap and discharging when it is expensive, rather than reacting passively to instantaneous bus voltage.

The control law is an inner current PI only. The current limits implemented are a maximum or minimum at ±0.60 A, a linear taper to 0 near the voltage limits, and a bus-health taper that reduces charging demand when the bus voltage sags, so the capacitor does not compete with the grid import module for current during a shared deficit.

4.2 Testing & Optimisation

The supercapacitor evaluation answers two questions: what is the round-trip energy efficiency η_{RT}, and how does it split into η_{charge} and $\eta_{\mathrm{discharge}}$? Both are answered from a single charge-and-discharge cycle, using the capacitor's known capacitance as an internal calibration reference. The same dataset supplies the efficiency figures the dispatch algorithm in Section 9 relies on.

4.2.1 Automated Cycle-Test Method

For a complete cycle from V_{low} to V_{high} and back, the firmware integrates bus-side input energy during charge,

$$E_{\mathrm{in}} = \int V_{\mathrm{bus}} \cdot I_{\mathrm{bus}} dt$$

and bus-side output energy during discharge,

$$E_{\mathrm{out}} = \int V_{\mathrm{bus}} \cdot |I_{\mathrm{bus}}| \; dt$$

The round-trip efficiency is then

$$\eta_{\mathrm{RT}} = \frac{E_{\mathrm{out}}}{E_{\mathrm{in}}}$$

This metric is preferred because both E_{in} and E_{out} are measured at the bus, so the result does not depend on estimating capacitor-side current. The capacitor-side stored energy $E_{\mathrm{cap}} = \frac{1}{2} C V_{\mathrm{cap}}^2$ (with $C = 0.5$F, two 0.25F capacitors in parallel) is still recorded before charge, after charge, after any hold phase, and after discharge. It is useful for checking the expected energy swing, but it is not used as the main efficiency metric because the terminal voltage relaxes after current steps due to ESR and charge redistribution.

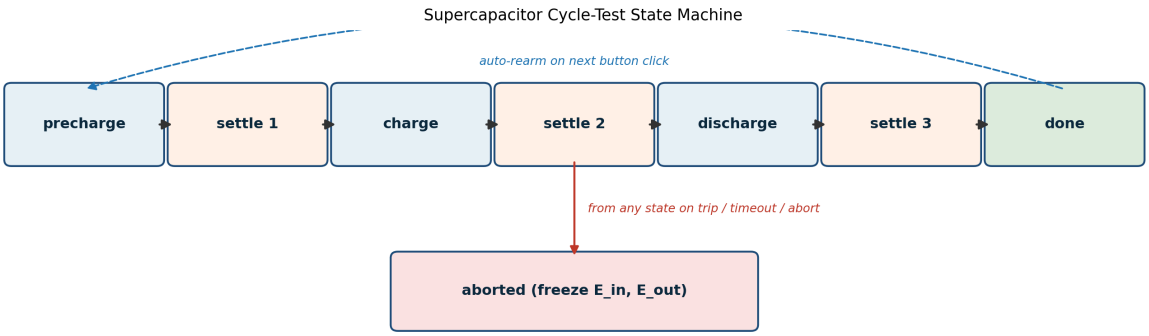


Figure 4.2: Supercapacitor cycle-test state machine.

As shown in Figure 4.2, the test is implemented as a firmware state machine triggered from the dashboard. It runs through precharge, settle 1, charge, settle 2, optional hold, discharge, settle 3 and done. Each settle phase records the open-circuit capacitor voltage only after a short delay, reducing the effect of the immediate ESR transient. Bus-side energy integration runs at the 1 kHz control tick, so the reported η_{RT} is not limited by the dashboard logging rate. The discharge endpoint is kept at $V_{\mathrm{low}} = 10.5\mathrm{V}$, matching the usable operating window, so any taper near the lower limit is included in the measured round-trip result.

Although η_{charge} and $\eta_{\mathrm{discharge}}$ can be estimated from the capacitor-side energy readings, they are not reported as final results. In practice, charge redistribution caused the discharge-side estimate to exceed 100% in some runs, which is physically impossible. The conclusion is that capacitor voltage is adequate for state-of-charge indication, but not for cleanly separating charge and discharge efficiency on this capacitor bank. The reliable metric carried forward is therefore the bus-side round-trip efficiency η_{RT}.

4.2.2 Round-Trip Efficiency Results

Two no-hold sweeps were run, with each operating point repeated three times. The first varied charge and discharge current from 0.1A to 0.6A at a 10V bus. The second varied bus voltage from 8.0V to 10.0V at 0.2A. Figure 4.3 shows a representative 10V, 0.2A cycle: the capacitor charges from 10.5V to the 15.7V cutoff, then discharges back to 10.5V. The measured bus-side energies are $E_{\mathrm{in}} = 38.2$ J and $E_{\mathrm{out}} = 33.7$ J, giving $\eta_{\mathrm{RT}} \approx 88\%$. The capacitor-side energy swing is approximately 34J, matching the design estimate from the usable voltage window.

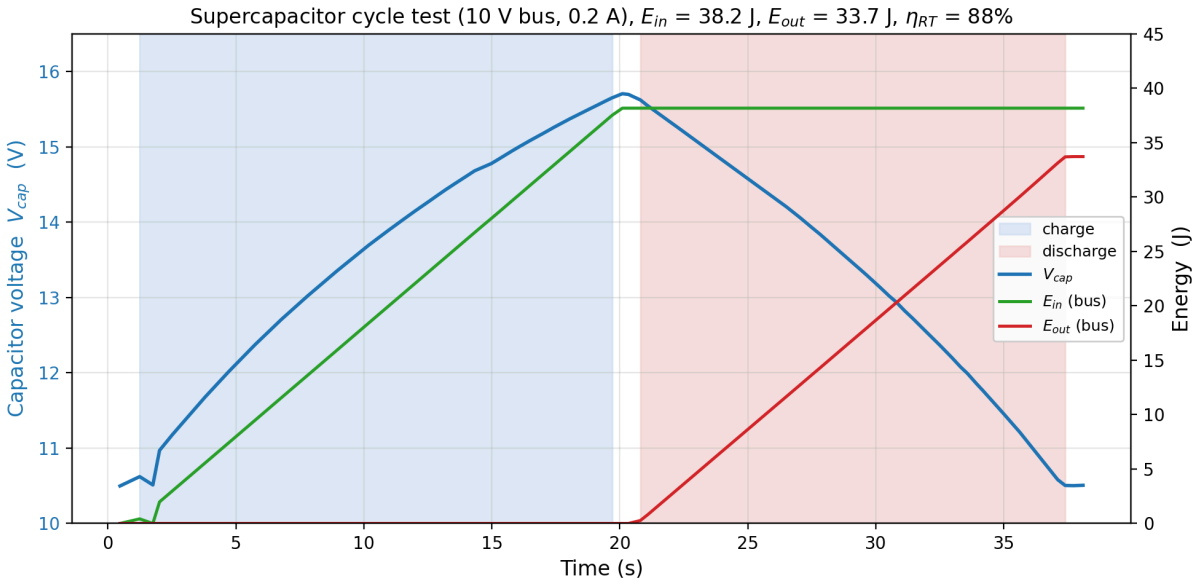


Figure 4.3: Measured supercapacitor charge and discharge cycle at 10V, 0.2A.

The measured round-trip efficiency is stable across the tested range. In the current sweep, η_{RT} falls only slightly from 89.7% at 0.1A to 86.9% at 0.5A, with the 0.6A result recovering to 87.4% within run-to-run scatter. This weak downward trend is consistent with conduction loss increasing with current. In the bus-voltage sweep, efficiency varies even less, rising mildly from 87.3% at 8.0V to 88.3% at 10.0V. This is consistent with the smaller boost ratio when the bus voltage is closer to the capacitor voltage.

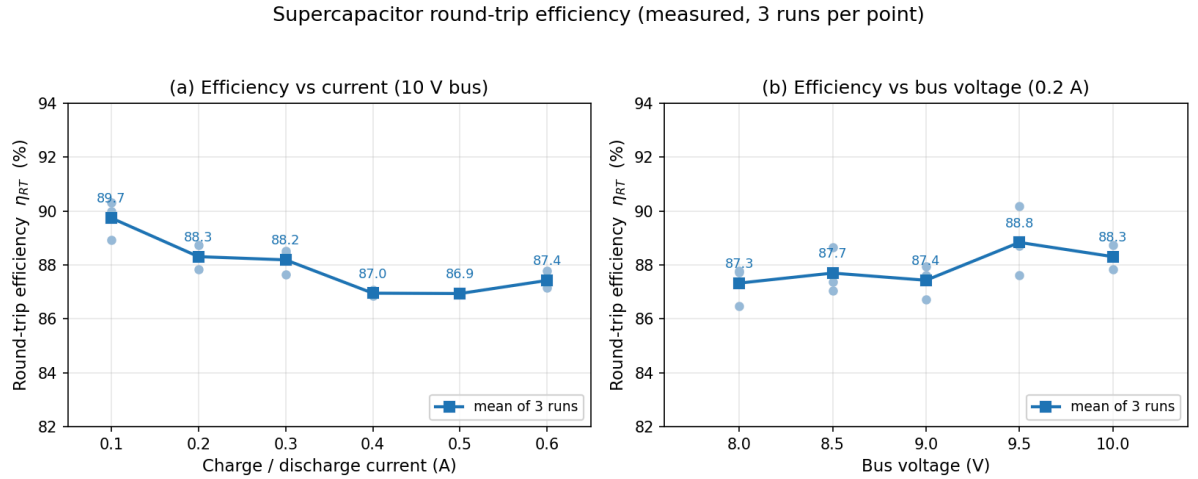


Figure 4.4: Measured round-trip efficiency versus current and versus bus voltage.

Charge current	Mean η_{RT}, 3 runs
0.1 A	89.7%
0.2 A	88.3%
0.3 A	88.2%
0.4 A	87.0%
0.5 A	86.9%
0.6 A	87.4%

Table 4.1: Supercapacitor round-trip efficiency by charge current at 10 V bus.

Bus voltage	Mean η_{RT}, 3 runs
8.0 V	87.3%
8.5 V	87.7%
9.0 V	87.4%
9.5 V	88.8%
10.0 V	88.3%

Table 4.2: Supercapacitor round-trip efficiency by bus voltage at 0.2 A.

The headline result is therefore that the supercapacitor path achieves approximately 88% immediate round-trip efficiency across the intended operating range. This figure is higher than the conservative 70 to 85% pre-test estimate and is sufficiently stable to use in the system-level dispatch model.

4.2.3 Self-Discharge and Hold-Duration Dependence

The same cycle test was extended with a zero-current hold phase between charge and discharge. Hold durations of 60, 120, 180, 240 and 300 seconds were tested at 0.2A and a 10V bus, with three repeats per point. This isolates the effect of storing energy in the capacitor before using it.

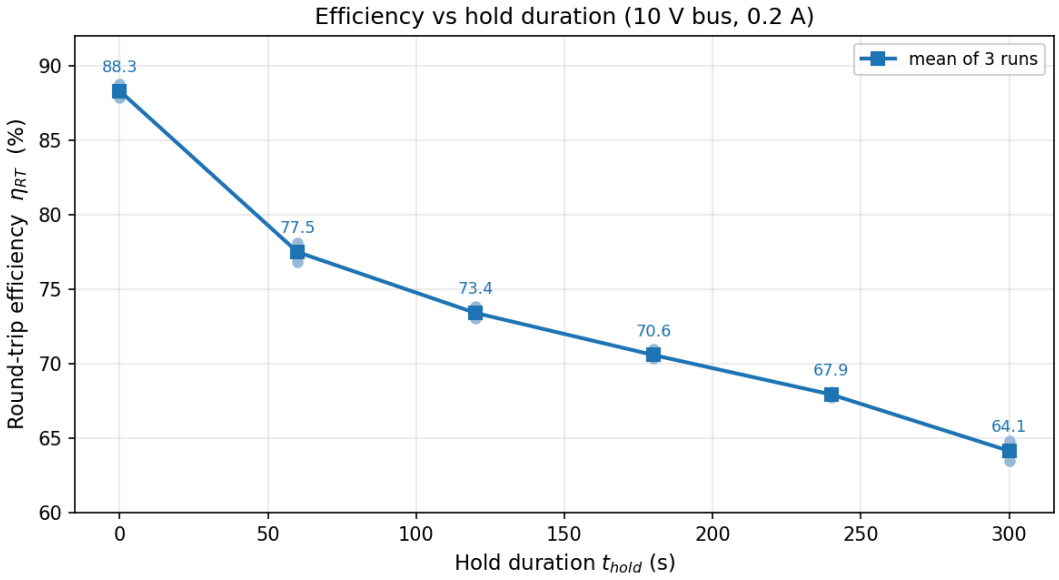


Figure 4.5: Round-trip efficiency versus zero-current hold duration.

Round-trip efficiency falls sharply with hold time: from 88.3% with no hold to 77.5% after 60 seconds, and to 64.1% after 300 seconds. Because $\eta_{\mathrm{RT}} = \frac{E_{\mathrm{out}}}{E_{\mathrm{in}}}$ is measured entirely from bus-side energy, this drop is a real loss of recoverable energy rather than an error in the capacitor-voltage measurement.

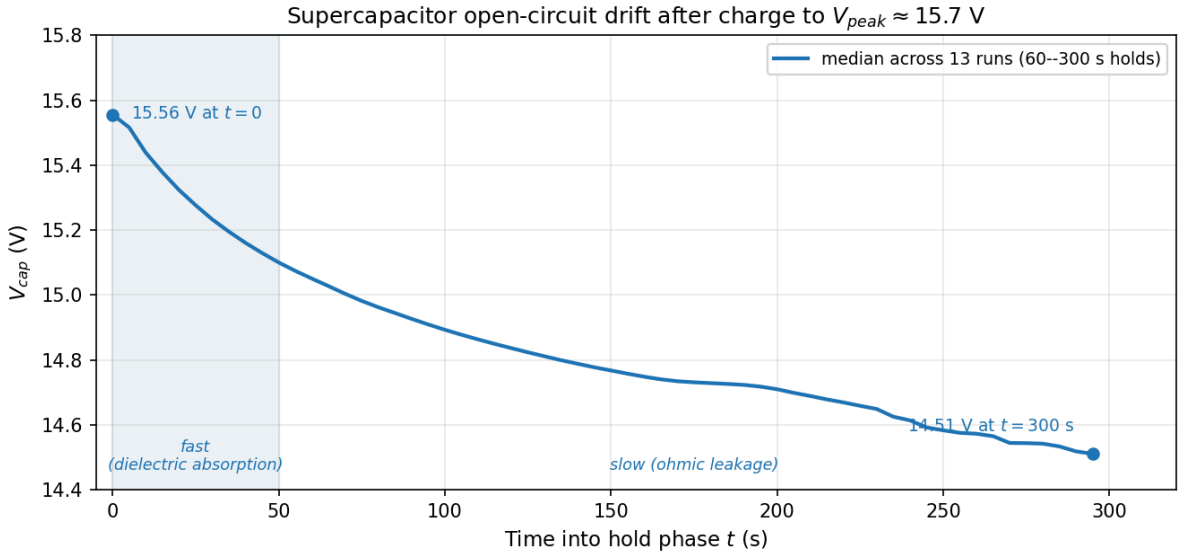


Figure 4.6: Open-circuit V_{cap} drift after charge: master curve across 13 hold runs.

Also as seen in Figure 4.6, after charging to the 15.7V cutoff, V_{cap} falls quickly by about 0.5V over the first 50 seconds, then continues to decay more slowly at roughly 2 to 3 mV/s. The fast component is attributed to charge redistribution within the supercapacitor [9], while the slower component is consistent with leakage. Since all hold-duration runs collapse onto the same relaxation curve, the loss can be treated as a repeatable function of hold time.

For dispatch this means the supercapacitor should be charged just-in-time rather than charged early and held near peak voltage. Immediate cycling gives about 88% round-trip efficiency, but a few minutes of hold time can erase much of the storage benefit. The economic algorithm should therefore delay charging until the stored energy is actually needed, unless the alternative is wasting PV energy that cannot otherwise be used.

5 Import & Export SMPS

5.1 Design & Simulation

5.1.1 Topology Choice

Both import and export modules are configured as synchronous buck converters with the BU/BO switch set to BUCK (the shared power stage is Figure 2.1). The import module places the 12V bench PSU on Port A and the 10V bus on Port B, sourcing current into the bus. The export module places the 10V bus on Port A and a resistor bank on Port B, stepping the voltage down across the resistor to dissipate energy. Current cannot be pushed into the PSU or bank, so the minimum reference current for both SMPS is 0. This satisfies the Port A above Port B rule.

The duty cycle is inverted for these SMPS compared to the PV and supercapacitor boost SMPS due to the inversion of the gate driver. Current sensing is performed by the INA219 driver across the 0.1 Ω Port B shunt. The choice of a dummy resistor is in accordance with the project brief [10]: using a resistor sized to sink the maximum export current instead of driving current into a bench PSU, which would collapse the terminal voltage.

The maximum export power is the sum of maximum PV power and capacitor power. The demo-day configuration of two parallel PV panels supplies a maximum of roughly 8W at the panel side, and the capacitor supplies a maximum of about 13W at peak 0.6A discharging current near the top of its window, making the maximum export power under 20W. Given that the buck SMPS has 10V at the input and the output voltage must be below that, the load bank needs a current of $\frac{P}{V} = \frac{20}{10} = 2$ A and a resistance of $\frac{V}{I} = \frac{10}{2} = 5\ \Omega$. The dummy load is therefore a 5 Ω bank and the inductor-current reference is capped at 2 A, the point at which the 10V and 2A limits coincide at the 20W corner, and which sits under the module's 2.8A firmware overcurrent trip.

5.1.2 Droop Setpoints: 9.9 V Import and 10.1 V Export

Each module runs a cascaded controller: an outer bus-voltage PI produces a reference current and an inner current PI produces a PWM which changes the voltage (Figure 5.1). The two modules differ only in the sign of the voltage error. The import module computes $v_{\text{err}} = V_{\text{target}} - V_{\text{bus}}$, so it sources more current the further the bus falls below its 9.9V setpoint. The export module computes $v_{\text{err}} = V_{\text{bus}} - V_{\text{target}}$, the sign flipped, so it dissipates more the further the bus rises above its 10.1V setpoint. Each reference current is limited to 2A on both modules (approximately 20W at the bus), the export value set by the worst-case surplus analysis in Section 5.1.1 and the import value by the grid interface rating.

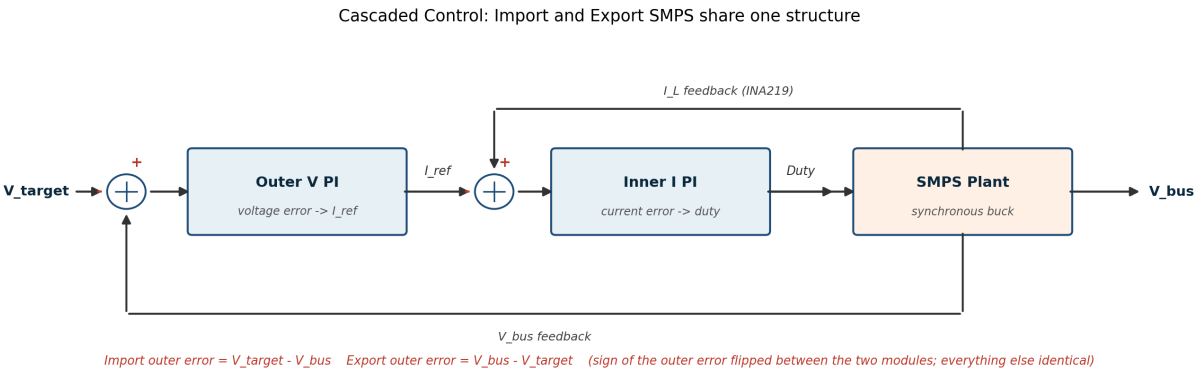


Figure 5.1: Cascaded control structure shared by Import and Export SMPS modules.

The dead-band that the symmetric setpoints create is analysed in Section 2.4. For implementation, a setpoint of 0.1V or below, or a disable command, is treated as fully off, which is how the bus changes from entirely import to entirely export.

5.2 Testing & Implementation

Both bus-facing modules received the same three-layer noise reduction as the PV module (Section 3.2), applied through the shared `INA219` class so import and export pick it up in one edit.

5.2.1 Anti-Windup and Soft-Start Grace

The outer voltage PI on each module uses clamping anti-windup, with the integrator bounded to the range that just saturates the current reference. At the default bound the integrator wound up far past saturation and took several seconds to unwind, observed as the rail fluctuating between 8V and 12V at about 1Hz when the PV and export SMPS ran together without the grid SMPS.

A 500ms soft-start grace disables the undervolt trip on every transition from inactive to active (enable, watchdog clear and trip reset), since the bus starts near 0V and would otherwise trip immediately; overvolt and overcurrent trips stay enabled throughout. The export module carries the same grace logic, with its output current clamped to 0 while the bus voltage is below target.

5.3 Evaluation & Optimisation

Under the dual-meter scoring of Section 2.3, import conversion loss costs money directly: the PSU reading is the billed quantity, so every 1% of loss inflates the effective import price per delivered bus Wh. Export loss instead appears as a gap between the dashboard's post-loss dissipation and the bus meter's pre-loss credit (Section 6.3.2 covers the same effect on the LED path; the fix in both cases is a current sensor between the bus and the SMPS). The import sweep is the characterisation that matters for cost and is the one reported here.

The import SMPS efficiency is $\eta_{\text{import}} = \frac{P_{\text{bus,delivered}}}{P_{\text{PSU,drawn}}}$. The pre-loss draw $P_{\text{PSU,drawn}} = V_{\text{PSU}} \cdot I_{\text{PSU}}$ is not measured on the import module because the shunt sits on Port B. Unlike the PV case in Section 3.3, I_{PSU} can be read from the bench PSU's built-in current display with manufacturer-stated accuracy (the same display the lecturer reads on demo day).

The bus-voltage axis is swept across nine points from 7V to 11V in roughly 0.5V steps, wider than the 9.9V to 10.1V dead-band so duty-cycle dependence is captured across the full operating

range. Currents of 0.1, 0.2 and 0.3A are used. At each of the 27 operating points the three-second-averaged $P_{\text{bus,delivered}}$ is recorded against $V_{\text{PSU}} \cdot I_{\text{PSU}}$ and the ratio taken.

V_{bus} (V)	Duty D	η at 0.1 A	η at 0.2 A	η at 0.3 A
7.0	0.58	77.8%	86.9%	89.3%
7.5	0.62	79.0%	87.3%	90.0%
8.1	0.67	81.1%	88.1%	90.4%
8.5	0.71	81.7%	88.4%	90.8%
9.0	0.75	82.1%	88.9%	91.2%
9.5	0.79	82.9%	89.6%	91.7%
10.0	0.83	83.8%	89.9%	92.5%
10.5	0.87	84.2%	90.6%	92.7%
11.0	0.92	84.7%	91.3%	93.6%

Table 5.1: Measured Import SMPS efficiency by bus voltage and load current.

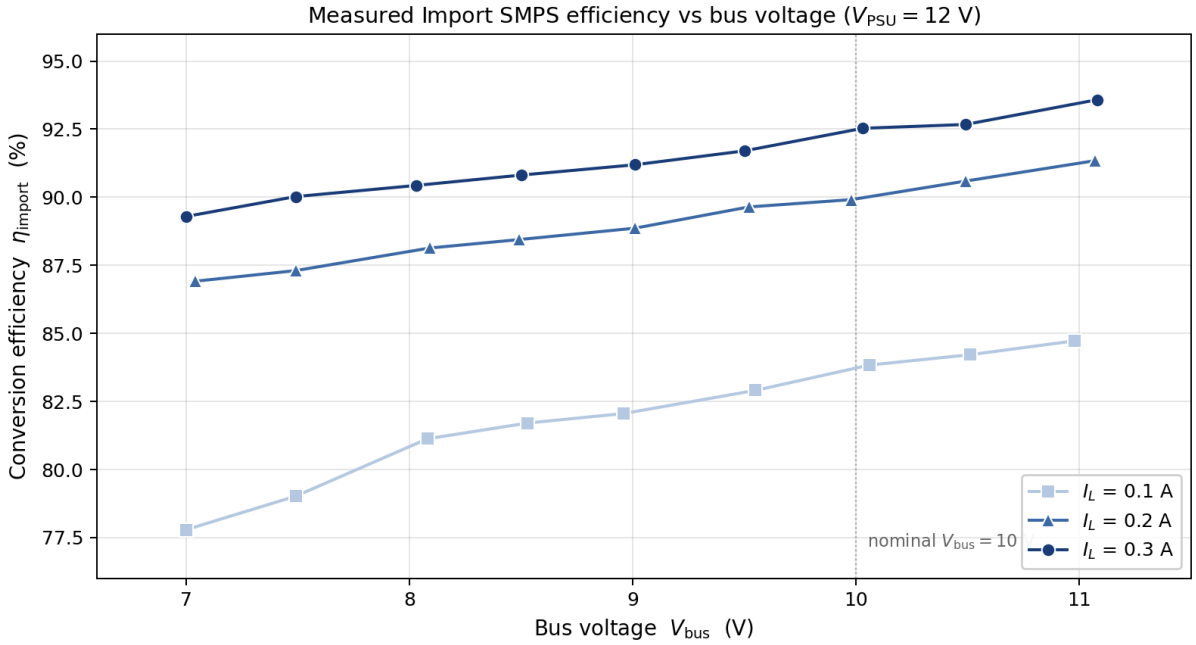


Figure 5.2: Measured Import SMPS efficiency versus bus voltage at three load currents.

At the nominal $V_{\text{bus}} = 10$V the converter delivers 83.8%, 89.9% and 92.5% at 0.1, 0.2 and 0.3A. Both trends (the rise with V_{bus} as duty approaches unity, and the rise with I_L as fixed switching overheads spread over more delivered power) match the buck loss model of Section 2.3 and confirm the converter is operating away from dropout. In cost terms, the 89.9% at the nominal 10V and 0.2A operating point inflates the effective import price by $\frac{1}{0.899} \approx 1.11$, an 11% premium absorbed by conversion loss that the dispatch algorithm of Section 9 sees directly through its PSU-referenced cost formula.

6 LED Load SMPS

6.1 Design & Simulation

6.1.1 Topology

The LED load is a triple driver assembly: three independent buck channels (red, yellow, blue), each a cut-down SMPS driving a 3W, 1A LED, all under one Pico W. Buck is the natural choice because the 3V LED forward voltage is well below the 10V bus. The hardware differs from the bidirectional modules in two ways the firmware must respect. Current is sensed by an external SPI ADC (MCP3208 family [11]) reading three current channels and three voltage channels through one chip select, with a 0.33 Ω shunt rather than 0.10 Ω, so the inductor current is $i_{\mathrm{L}} = 3 \cdot V_{\mathrm{shunt}}$. Each channel has a dedicated enable pin that must be driven high on every tick; the gate driver gates the PWM off whenever its enable is low. The LED forward voltage is reconstructed from the two ADC reads as $v_{\mathrm{LED}} = 2 \cdot v_{\mathrm{v,pin}} - v_{\mathrm{i,pin}}$, correcting for the sense divider and shunt drop. One of the three identical channels is shown in Figure 6.1.

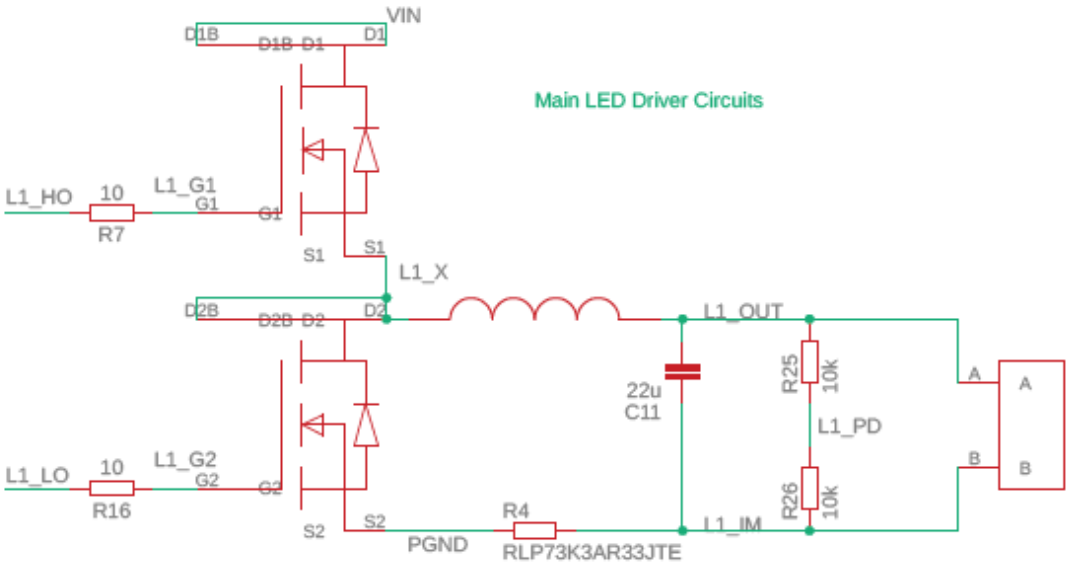


Figure 6.1: LED buck driver power stage, one of three identical channels [12].

6.1.2 Control Mode: CC vs Commanded Power

An LED behaves as a fixed-voltage device, so its power is set by controlling current. The web server hands the system a power figure, so each channel runs commanded-power mode through constant current: power is divided by measured forward voltage to give a current reference $i_{\mathrm{ref}} = \frac{P_{\mathrm{demand}}}{v_{\mathrm{LED}}}$, which a per-channel current PI tracks. The reference is clamped to channel ratings of 3.0W and 1.05A, and a forward-voltage floor guards the division when a string is dark. When demanded power is zero the enable pin is dropped and the PI is reset to stop the integrator winding up while the channel is off.

6.2 Testing & Implementation

The example driver firmware was reworked to drop its external PID dependency in favour of the same inline PI used across the bidirectional modules, so the same controller structure is used throughout the project. The per-channel gains and duty saturation range differ from the

bidirectional modules to suit the 0.33 Ω shunt and the driver's own PWM limits. Safety is per-LED: if any channel's current exceeds 1.4 times its rated maximum the firmware latches a trip and records which colour tripped, so a faulty LED is diagnosable from the logged readings instead of showing up as a generic fault.

A startup check helped separate wiring faults from real ones. The firmware reads every SPI ADC channel a few times before any PWM runs; if a current reading already looks too high at this point, the cause must be a wiring or measurement problem (a floating ADC input, a wrong port, or a low bus voltage) rather than a real LED overcurrent, and the printed values show which. Shutdown order matters as well: the enable pins are switched off before the duty cycles are zeroed, and since the enable pin cuts the PWM at the driver itself, the LEDs turn off immediately. A 10-second watchdog that switches all channels off when commands stop arriving exists in the code but is currently disabled (Section 8.3); instead, the operator can press Stop on the dashboard, which sends a zero-power command to every LED channel.

6.3 Evaluation & Optimisation

This section reports the per-channel power-following accuracy (which verifies Requirement 1), then explains why conversion efficiency is not separately characterised.

6.3.1 Power-Following Accuracy

A 60-second scripted test exercises each channel individually and then together, with commanded and measured per-channel powers logged from the dashboard at 20 Hz. Figure 6.2 overlays the two: the faint trace is commanded power, the dark trace is measured power delivered to the LED string. The sequence is red at 2W for 6s, off, yellow at 2W for 6s, off, blue at 2W for 6s, off, all three together at 1, 2 and 3W for 10s, off, and a 15-second simultaneous ramp from 0 to 3W.

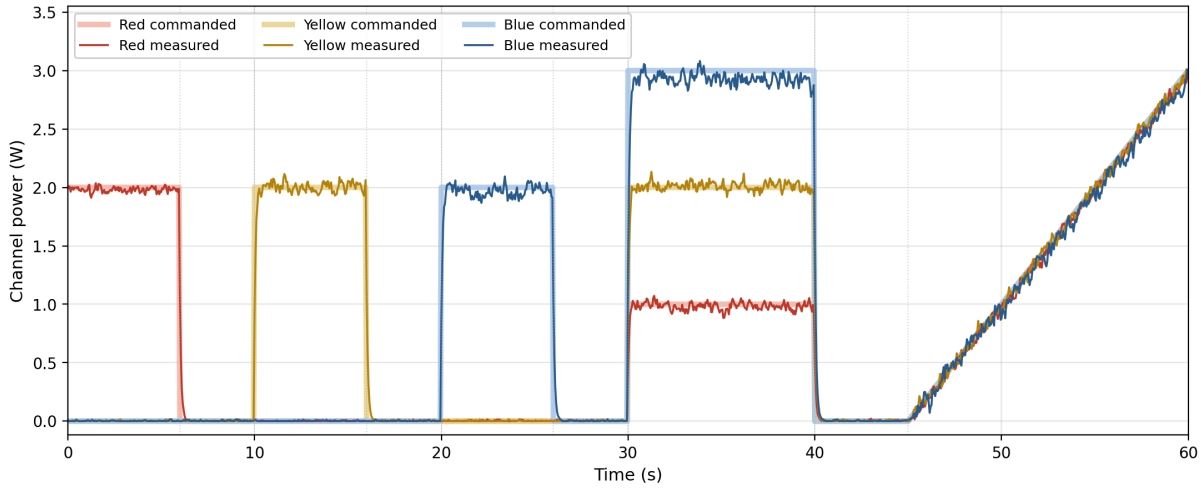


Figure 6.2: Per-channel commanded and measured LED power across the 60-second tracking sequence.

Each channel tracks within about ±3% across all operating points, with the residual driven by per-string forward-voltage variation that the $i_{\text{ref}} = \frac{P_{\text{demand}}}{v_{\text{LED}}}$ division (Section 6.1.2) cannot compensate when v_{LED} is itself estimated through the SPI ADC chain. Step settling on each on/off transition resolves inside one 50 ms dashboard sample. The all-on and ramp phases confirm channels do not interact through the shared bus or SPI ADC, so a multi-colour demand is served additively.

6.3.2 Bus-Side Measurement Gap

The LED Load conversion path sits between the bus meter and the LED string, in the same position in the power path as Export. The lecturer's bus meter registers $P_{\text{LED,bus}} = P_{\text{LED,channel}} + P_{\text{losses}}$ as load, while firmware reports only the per-channel post-loss values p_{red}, p_{yel}, p_{blu} measured at the LED string. As a result, the dispatch algorithm reads an LED demand under-reported by exactly the LED Load buck conversion loss, and the same effect applies on Export where the post-loss dissipation under-reports the bus-credited revenue (Section 5.3). The buck loss could in principle be attributed to either side of the converter, but with no instrumentation between the bus and the SMPS we cannot measure where it actually lands. The practical option is to maximise η_{LED}: at $\eta_{\text{LED}} = 1$ the post-loss value would equal the bus value and dispatch decisions would align exactly with the lecturer's reading; at realistic efficiencies the algorithm runs on slightly optimistic numbers and PnL is marginally worse than a perfectly-informed policy would achieve. The proper fix is a dedicated INA219 between the bus and the SMPS, feeding the true bus-side current to the dashboard so dispatch operates on the lecturer's view of the load; this was identified but not implemented within the project window.

7 Measurement Filtering

Noise on the power readings was bad, obscuring the operating-point comparisons Sections 3.3, 4.2 and 5.3 depend on. Typical oscillation on P_{panel} was several hundred milliwatts peak-to-peak against a steady-state mean of a few watts, with similar relative magnitudes on the other modules. The fix is a three-layer attenuation applied identically across the PV, supercapacitor, import and export modules.

7.1 Hardware Averaging

The bidirectional template inherits an INA219 CONFIG register of 0x199F (12-bit, single-sample, 532 µs conversion [13]), which lets most of the switching ripple through unaveraged on the current-sense shunt. Reprogramming to 0x1E67 (16-sample averaging, 8.51 ms conversion) across all four modules clears this, in which the longer conversion time is still over twenty times faster than the slowest downstream consumer, the supercapacitor's 200 ms OCV window (Section 4.2), so no process is kept waiting. PV and supercapacitor pick this up through identical edits to their inline `ina_init` functions, and import and export through a single edit to the shared `INA219` class.

Voltage channels are read by the Pico's onboard 12-bit ADC [14] through a resistive divider and vary by several LSBs sample-to-sample regardless of INA219 settings. Each `read_va` and `read_vb` call performs four single reads and right-shifts the sum by two to take the mean:

$$V_{\text{oversampled}} = \frac{V[0] + V[1] + V[2] + V[3]}{4}$$

Cost per call is about 10 µs, well inside the 1 ms control tick budget. Peak-to-peak jitter falls by roughly 1.6×, with diminishing returns past four samples.

7.2 Exponential Moving Average

A first-order EMA is applied in software on top of the hardware-averaged readings:

$$V_{\text{filt}}[n] = (1 - \alpha) \cdot V_{\text{filt}}[n-1] + \alpha \cdot V[n]$$

with $\alpha = 0.05$ at the 1 kHz sample rate, giving a time constant $\tau \approx 20$ ms and a corner frequency near 8 Hz. The filtered signals drive the dashboard display, the three-second block averages, the P&O power comparison (Section 3.2) and the cycle-test energy integrators (Section 4.2). $\alpha = 0.05$ is bounded both ways: too small (longer τ) and the filter adds phase lag that slows displayed transients on dashboard reads; too large (shorter τ) and it passes switching-ripple aliasing through. 20 ms gives about 60 ms to 95% step settling, which fits inside the 150 ms second-half averaging window of the P&O dwell and the 200 ms tail of the cycle-test settle phases. Figure 7.1 shows the response: a corner frequency near 8 Hz that rejects aliased ripple, with a step settling inside 60 ms so the displayed transient is barely delayed.

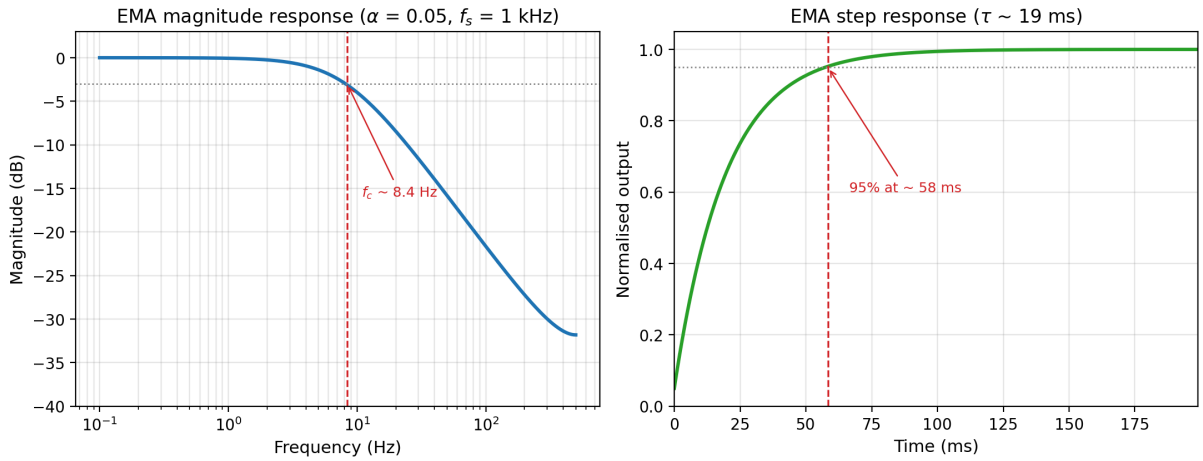


Figure 7.1: Measurement EMA magnitude and step response at alpha = 0.05.

The inner PI loops and safety guards bypass the software EMA and block-average outputs, mainly for two reasons. First, adding 20 ms of phase lag to a current PI tuned against unfiltered inputs would shift its stability margin and force a retune. Second, the trip thresholds (panel undervolt, bus overvolt, overcurrent) were designed for instantaneous response; a filtered safety guard would react 20 ms late, allowing more energy into a fault before tripping. Keeping the filter on the display and characterisation paths only preserves control-loop behaviour and trip response while still giving the operator stable readings. Note that the current readings still include the INA219′s internal conversion averaging, but no additional software filtering is inserted into the control path.

7.3 Three-Second Block Average

Each module computes a true block average of its bus-side power over a three-second window in firmware:

$$P_{\text{avg3s}} = \left(\frac{1}{N}\right) \sum_{i=1}^{N} P[i]$$

with $N = 3000$ at 1 kHz. The window restarts on changes to the commanded operating point (MPPT mode for PV, I_{cmd} for supercapacitor, V_{target} and I_{max} for import and export), so each completed average reflects one operating point. Rounding the operating-point values before comparison avoids spurious resets from tiny floating-point differences introduced by JSON parsing. The computed average lives in each module's data dictionary; persistence into the backend SQLite store and rendering on the operator dashboard are scheduled wiring work, so the side-by-side comparisons in Sections 3.3 and 5.3 read the filtered live readings directly. Inside the supercapacitor cycle test the block average sits on top of phase changes as a real-time sanity check that the commanded current has settled.

7.4 Evaluation

The three-layer treatment is evaluated by the noise reduction it delivers on the per-module bus-side power readings. Figure 7.2 overlays a raw single-sample trace against the same operating point after the three-layer filter, shifted to a 2W PV operating point for visual context.

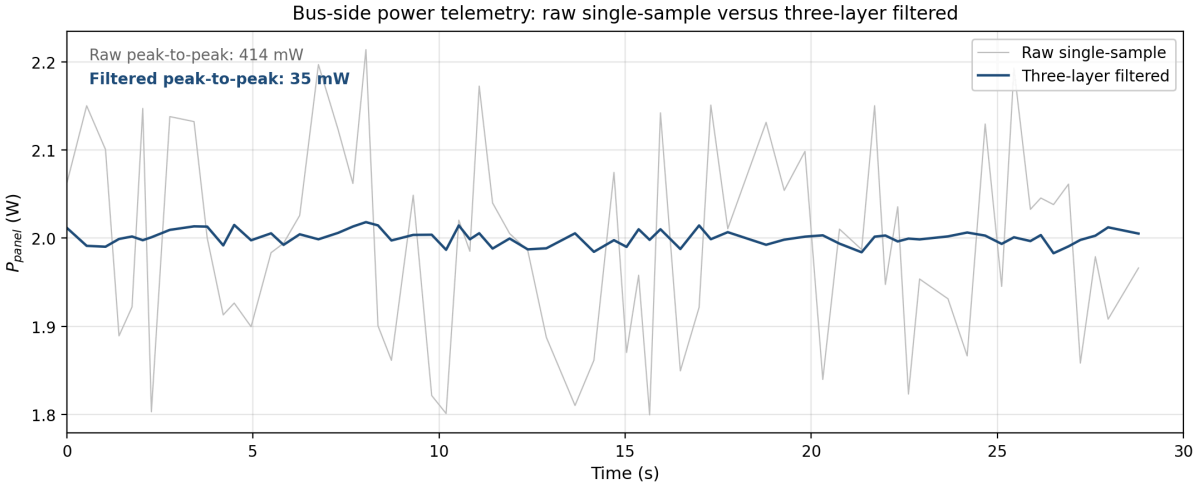


Figure 7.2: Bus-side power readings: raw single-sample versus three-layer filtered.

Raw P_{panel} oscillation falls from about 300 mW peak-to-peak at the 1 kHz tick to under 10 mW per P&O dwell-averaged comparison (Section 3.2), roughly a 30× reduction. Equivalent attenuation on the supercapacitor, import and export modules brings the three-second-averaged readings inside a few milliwatts, fine enough to resolve the 1.4 percentage-point irradiance-to-irradiance spread in the PV efficiency sweep (Section 3.3) and the 1 to 2 percentage-point steps between adjacent supercapacitor charge currents in the round-trip sweep (Section 4.2).

8 Software System

The software system provides the communication, control and monitoring infrastructure for the smart-grid platform. It links the Azure simulation server, backend services, embedded modules and operator dashboard into a unified system.

8.1 Architecture

The lowest tier consists of firmware running on each Pico-controlled module. Every module runs a lightweight HTTP server on core 1 and exposes a JSON `POST /cmd` endpoint for incoming setpoints. Keeping networking on core 1 isolates socket handling from the 1 kHz control loop on core 0, avoiding control jitter. The same thread also uploads telemetry to the backend with role-specific `POST /modules/tlm/{role}` requests (1s periodic heartbeat plus command-triggered pushes). The 1s heartbeat is chosen as a monitoring compromise: it is short enough to surface faults, voltage excursions and thermal issues before they escalate, yet slow enough to keep WiFi airtime and backend write load modest across all modules, with command-triggered pushes providing near-immediate visibility after setpoint changes.

The middle tier is a Python FastAPI backend with a SQLite store. It is the only bridge between browser clients and modules: commands enter FastAPI first, then FastAPI forwards them to the addressed Pico module; telemetry arrives from modules, is timestamped and persisted, and is then served to the frontend as latest and historical snapshots. A standalone poller ingests Azure simulation data and writes one deduplicated grid snapshot per new `(day, tick)`.

The top tier is a React single-page application built with Vite. The frontend provides monitoring, visualisation and operator controls through a browser interface.

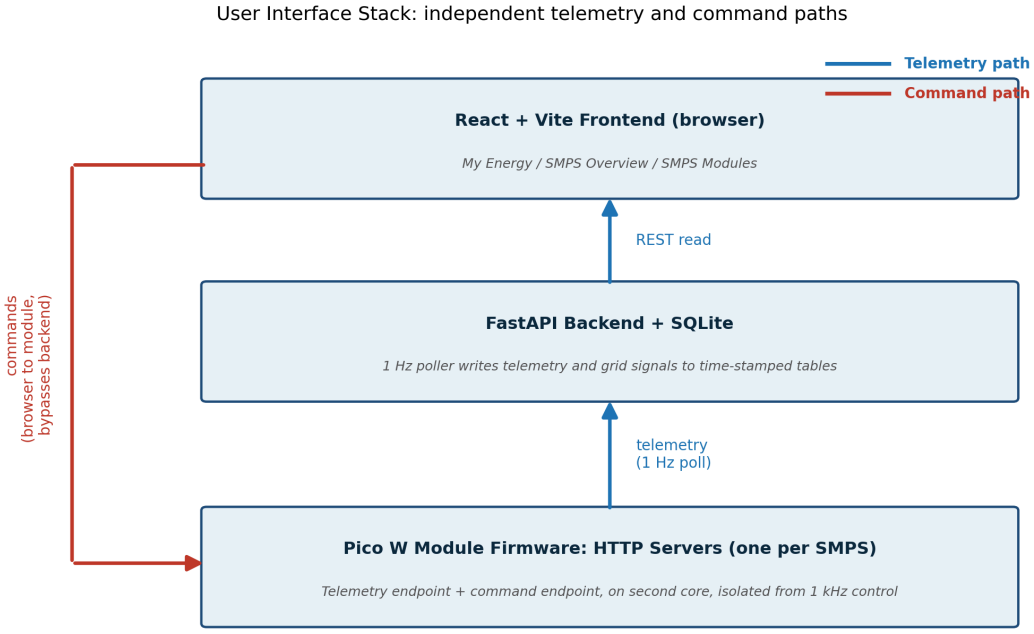


Figure 8.1: Software stack showing module readings flowing upward to FastAPI and control commands flowing downward through the same backend.

The overall architecture is summarised in Figure 8.1. Telemetry is push-based from modules to backend (`/modules/tlm/{role}`), while operator and EMS commands are backend-mediated (`/modules/cmd/{role}` to FastAPI, then forwarded to the corresponding Pico module `/cmd`).

8.2 Backend and Control Layer

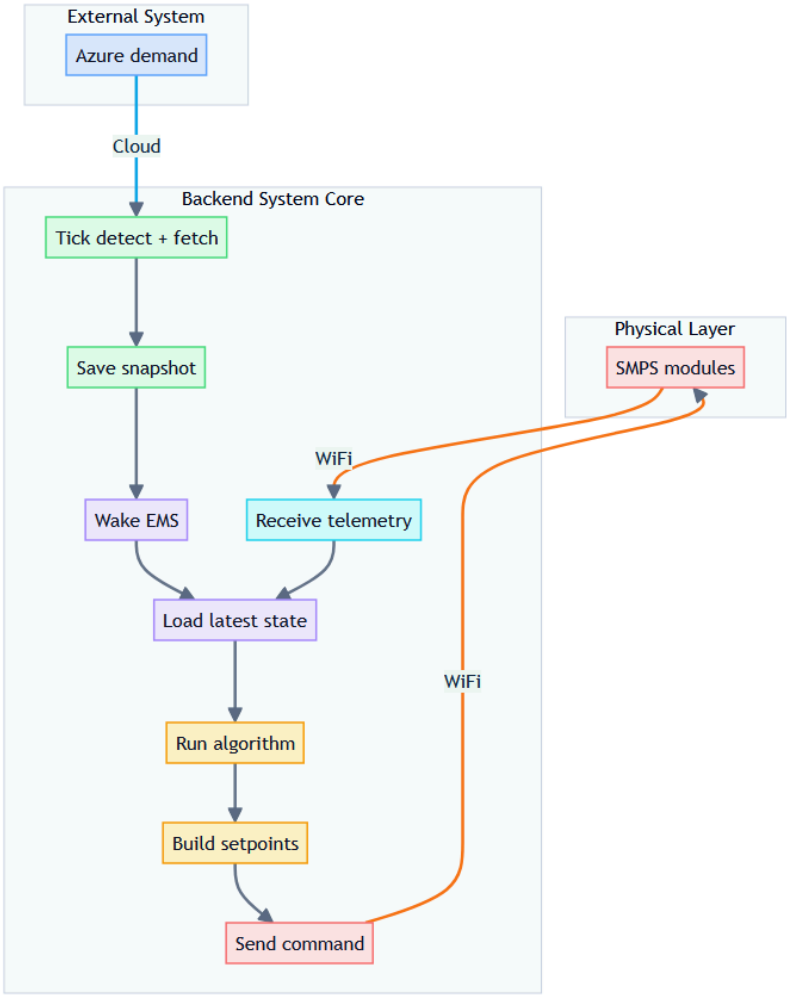


Figure 8.2: EMS control flow from Azure tick ingestion through FastAPI-mediated command and telemetry paths.

The backend is the central coordination layer. In addition to storing module telemetry and grid snapshots, it hosts the Energy Management System (EMS) loop that reads the latest committed state, runs the selected algorithm, and dispatches setpoints to LED, supercapacitor, grid and export modules via the command-proxy path.

The Azure poller checks `/demand` at 1 Hz and treats `(day, tick)` as the change detector. Only when a new tick appears does it fetch `/sun`, `/price` and `/deferables`, save one grid snapshot, and notify FastAPI through an internal event endpoint. The EMS auto-loop wakes on this event (with a fallback timer), deduplicates already-applied ticks, and skips stale or tripped conditions before dispatching.

Algorithms are registered and switchable at runtime from the dashboard. Decisions, telemetry snapshots and grid snapshots are logged in the same store, enabling full replay and offline comparison of baseline and improved policies.

The backend also provides run-management utilities, including auto-run control, algorithm selection, CSV export and run-scoped P&L summaries for post-test analysis.

8.3 Frontend Dashboard

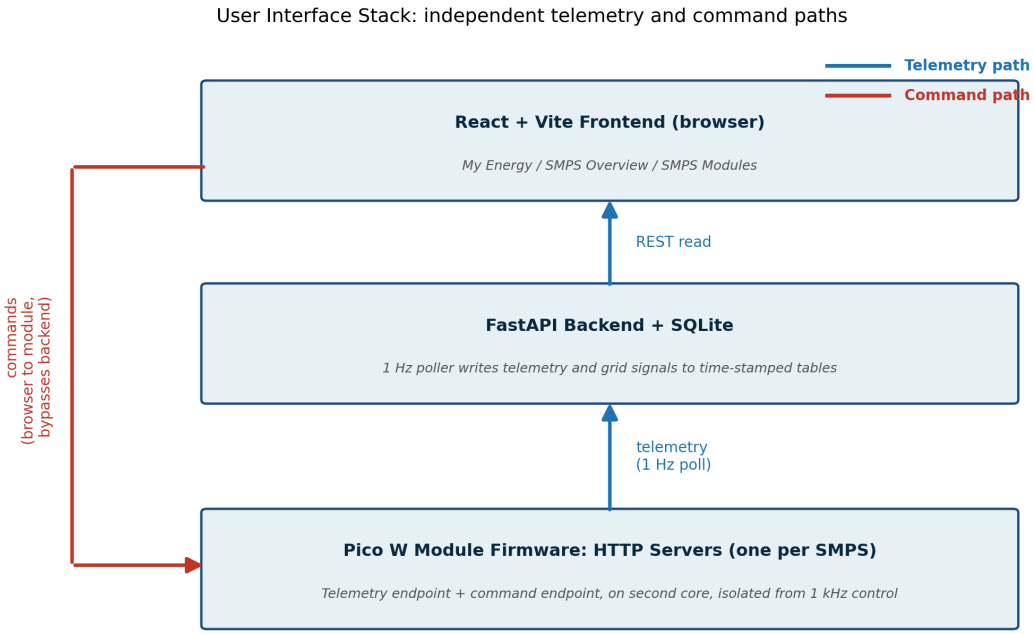


Figure 8.3: Frontend role in the communication path: all reads and writes go through FastAPI rather than direct browser-to-Pico links.

Three operator views are provided. The default **My Energy** dashboard presents a high-level overview of the microgrid, including current demand, solar irradiance, energy prices, active deferrable loads and module health indicators. Historical demand and irradiance data are displayed as charts, while a status panel highlights whether each module is online, stale, tripped, disabled or in a watchdog state.

The **SMPS Overview** page focuses on system-level monitoring. Historical measurements from the PV, grid, export, supercapacitor and LED modules is visualised using time-series plots of voltages, currents and power flows. This view is intended for observing overall energy movement through the system and diagnosing interactions between subsystems.

The **SMPS Modules** page provides detailed engineering controls. Each module is represented by a dedicated card showing live readings, recent history and module-specific controls. Operators can configure MPPT modes on the PV converter, adjust bus-voltage targets for grid and export modules, command supercapacitor charge or discharge currents, set LED load power levels and reset protection trips. These actions hit FastAPI endpoints, which then forward module commands to the relevant Pico. The same interface also provides algorithm selection, automatic EMS execution controls, run-scoped profit-and-loss summaries and data export tools.

8.4 Design Trade-offs

Several implementation choices were revised during integration to improve timing reliability. The first version of the Azure poller attempted to fetch the full grid state at a fixed five-second interval. In practice, a complete cycle of `/demand`, `/sun`, `/price` and `/deferables` requests could take longer than the Azure update period, causing occasional missed ticks. The final design instead checks only `/demand` once per second and treats `(day, tick)` as the change detector. The remaining endpoints are fetched only when a new tick is observed, and are requested in

parallel. This keeps tick-detection latency below one second while avoiding unnecessary repeated requests to the slower auxiliary endpoints.

Data collection was changed for a similar reason. Early versions used backend polling of module telemetry endpoints. In practice, temporarily offline modules caused repeated timeout handling and poor scalability. The final design is push-based: each module uploads its telemetry to FastAPI, where it is persisted and served to clients. This removes backend wait-on-timeout polling overhead and scales better as module count grows. The trade-off is that data freshness now depends on each module's own uplink health.

Control commands were kept backend-mediated rather than being sent directly from the frontend to the Pico modules. This allows the EMS algorithm to run headlessly on the backend and issue commands without requiring the dashboard to remain open. It also provides a single control interface for both automated EMS actions and manual operator commands. Centralising command dispatch in the backend ensures that all control actions are logged alongside module readings and grid-state data, enabling consistent run replay, debugging and post-experiment analysis. In addition, module addressing and communication logic are implemented in one location rather than being duplicated across multiple frontend views or external clients.

8.5 On-Module Fallback Dashboard

As a final redundancy, a single-file dashboard is bundled into module firmware and served at the module root path. This provides direct local control of a single module through its on-device HTTP server during bench bring-up, even if the backend stack is unavailable. In normal operation, however, project telemetry logging, EMS automation and multi-module coordination all depend on the backend-mediated path described above.

9 Algorithm Software

The per-module loops of Sections 3 to 6 keep the bus stable; this section covers the coordinator that decides what each module should be doing. The coordinator runs one of two policies: a reactive naive baseline that the brief asks the smart policy to beat, and a forecast-aware Smart V1 that schedules deferrable loads into cheap ticks and charges the supercapacitor cheaply to discharge it into the evening price spike. Both are evaluated on the same grid under the same scripted day, where the smart policy delivers a 20% net-cost saving averaged across two lab runs, and offline across 1000 simulated days, where it saves a mean of 28% and wins on every day.

9.1 Coordinator Loop and Economic Objective

The coordinator is a background thread inside the FastAPI backend, woken whenever the poller commits a new Azure tick (with a 5-second fallback timer in case a wake-up is missed). On each pass it reads the latest grid snapshot and the latest PV and supercapacitor readings from the database, evaluates the active algorithm's `decide(state)` function, and dispatches the resulting commands to the four actuators (LED Load, supercapacitor, grid, export) in that fixed order. The loop deduplicates against the Azure tick so the same tick is never applied twice, skips any tick where the grid snapshot is older than 15 seconds, and refuses to dispatch to a module currently latched in a trip. Because each module's inner loop holds the last commanded setpoint autonomously, a coordinator stall cannot destabilise the bus: droop, PI control and per-module clamps run independently at 1 kHz.

The objective is to minimise the net cost of grid energy over a daily Azure cycle while always meeting demand. Following the metering setup of Section 2.3, import is debited at the PSU reading and export credited at the bus reading, so the per-day cost is

$$C = \sum_t \big(P_{\mathrm{PSU}}(t) \cdot p_{\mathrm{sell}}(t) - P_{\mathrm{export,bus}}(t) \cdot p_{\mathrm{buy}}(t)\big) \cdot \Delta t$$

with $P_{\mathrm{PSU}}(t) = P_{\mathrm{import,bus}} \frac{t}{\eta_{\mathrm{import}}}(V_{\mathrm{bus}}, I_L)$ coupling the import draw to the efficiency map of Section 5.3. Prices follow the grid's perspective: p_{sell} is what the utility charges us on imports, p_{buy} what it credits us on exports, with $p_{\mathrm{buy}} \approx 0.5 \cdot p_{\mathrm{sell}}$ at every tick. That spread is the main opportunity the algorithm can exploit. The coordinator's available actions are deferrable scheduling, supercapacitor charge or discharge, and net import or export, subject to instantaneous demand being met, each deferrable receiving its full energy inside its window, and the supercapacitor and grid modules staying inside their hardware envelopes.

9.2 Algorithm Structure

Each policy is a single `decide(state: GridState) -> Commands` function with no internal state, registered with an in-process registry; the operator switches the active policy at runtime from the dashboard and the choice is persisted to disk. `GridState` carries the Azure tick, base demand, PV panel power, bus voltage from the supercapacitor module, supercapacitor terminal voltage, the live current-tick prices and the full deferrable list. `Commands` carries the four actuator payloads plus a diagnostics block (regime, PV derate, effective load, net imbalance, supercapacitor headroom, per-deferrable rates) that the dashboard renders for whichever policy is active. Adding a policy is a register call plus one module.

One detail matters for the live path. The Azure feed republishes each deferrable's full original energy at every tick of its window, so a policy that treats `d.energy` as remaining joules would

re-allocate the same load over and over. A small served-tracker module accumulates the bus-side energy actually dispatched per deferrable, credits only after a successful LED command (a dropped command does not count as served energy), and subtracts the running total from `d.energy` before the policy sees it. Without the tracker the LED was over-driven by 233 J on a 615 J target, a 38% over-dissipation; with it, the commanded LED energy lands within 7 J of target on both naive runs and within 41 J on the smart runs, where the residual falls out as export (the LED clamp at 9 W spills surplus into the bus).

9.3 Naive Baseline

The naive policy is a five-step reactive symmetric-droop rule. PV is first derated to bus side by a conservative $\eta_{\mathrm{PV}} = 0.90$ (the measured value in Section 3.3 is 96 to 98%, so the bias keeps the algorithm on the surplus side of modelling error). Active deferrables are then spread linearly across their remaining real-time window so the per-tick rate is $\frac{E_{\text{remaining}}}{5 \cdot N_{\text{ticks remaining}}}$ watts, with adaptive catch-up baked in: under-serving last tick raises the next tick's rate. Effective load is base demand plus the sum of those rates and is commanded to the LED Load, split three ways across red, yellow and blue channels with each firmware-clamped to 3 W. The net imbalance $P_{\mathrm{net}} = P_{\mathrm{PV,bus}} - P_{\mathrm{load,eff}}$ is then offered to the supercapacitor through a 1 W deadband (noise floor of the bus), with charge current $\frac{P_{\mathrm{net}}}{V_{\mathrm{bus}}}$ clamped to 0.6 A and refused above 15.7 V, discharge current $|P_{\mathrm{net}}| \frac{}{V_{\mathrm{bus}} \cdot \eta_{\mathrm{cap}}}$ refused below 10.5 V and linearly tapered through the 10.5 to 11 V firmware soft region. Anything the supercapacitor cannot absorb spills to droop: the grid and export modules sit permanently enabled at 9.9 V and 10.1 V respectively and passively make up the residual.

Figure 9.1 shows the naive policy across one Azure day on the grid. The cap power panel is small and twitchy, charging only when noise pushes the bus above the deadband; the grid panel does almost all of the work, importing through the morning, evening and night and exporting only the small midday surplus that the cap could not absorb.

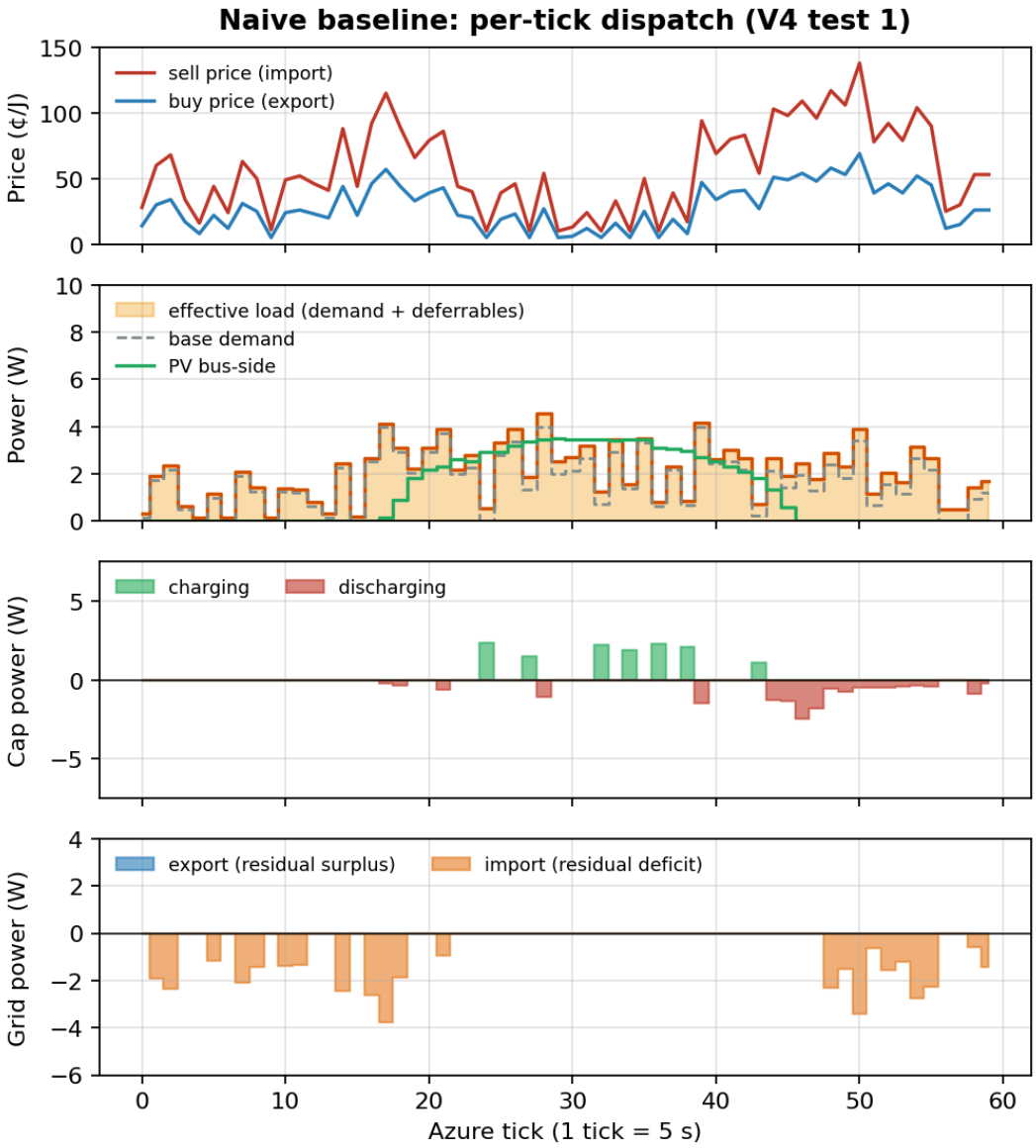


Figure 9.1: Naive baseline: per-tick dispatch over one Azure day (V4 test 1).

9.4 Smart V1: Forecast-Aware Scheduling and Arbitrage

Smart V1 keeps every safety guard of the naive policy (LED clamp, supercapacitor taper, 0.6 A clamp, deadband, discharge-side η_{cap} compensation) and replaces two of its reactive steps with forecast-aware ones. The forecast itself is closed-form: the Azure simulation has a deterministic mean (sun follows a sine over ticks 15 to 45, base demand follows a piecewise-linear envelope, sell price = max(10, 10 + base − sun)), with Gaussian noise on top. Smart V1 evaluates that mean per tick rather than fitting historical data.

The first replacement is the deferrable scheduler. Instead of spreading each deferrable across its window, future ticks are ranked by marginal cost-to-serve: a tick in forecast PV surplus costs p_{buy} (the foregone export revenue), a tick in deficit costs p_{sell} (the imported energy). The scheduler fills the cheapest ticks first, respecting the LED's 9 W per-tick capacity and giving the tightest-deadline deferrable first pick. The second replacement is the supercapacitor dispatcher. At every tick it compares the current sell price against the best future sell price across the rest of the day, after subtracting expected leakage, using a piecewise leakage model

fitted to Section 4.2.3 (a 12% dielectric step over the first minute, then 0.25% per Azure tick). It picks one of six actions: an end-of-day drain past tick 55, a discharge when the current price beats the best future price by more than 20 ¢/J, a charge when the forecast return exceeds the charging cost by the same margin, an import-and-charge action that buys grid energy when a strong forecast peak exceeds the current price by more than 50 ¢/J, an empty-cap refill rule that lowers the import bar when a forecast peak is within eight ticks, and a fall-back passive rule that mirrors the naive deadband when none of these triggers fire. Two pre-peak guards (refuse passive discharge below 70 ¢/J after tick 42, and a fixed price floor instead of forecast comparison) were added after a lab trace showed the supercapacitor being drained at moderate prices and arriving empty at the real evening peak.

Figure 9.2 shows the smart policy on the same day. The supercapacitor panel now has a large midday charge burst (ticks 24 to 32, where PV surplus and cheap prices coincide) and an evening discharge burst (ticks 45 to 50, where the sell price rises above 100 ¢/J). The grid panel imports most of its energy in the cheap morning and exports the midday surplus that exceeds the LED clamp.

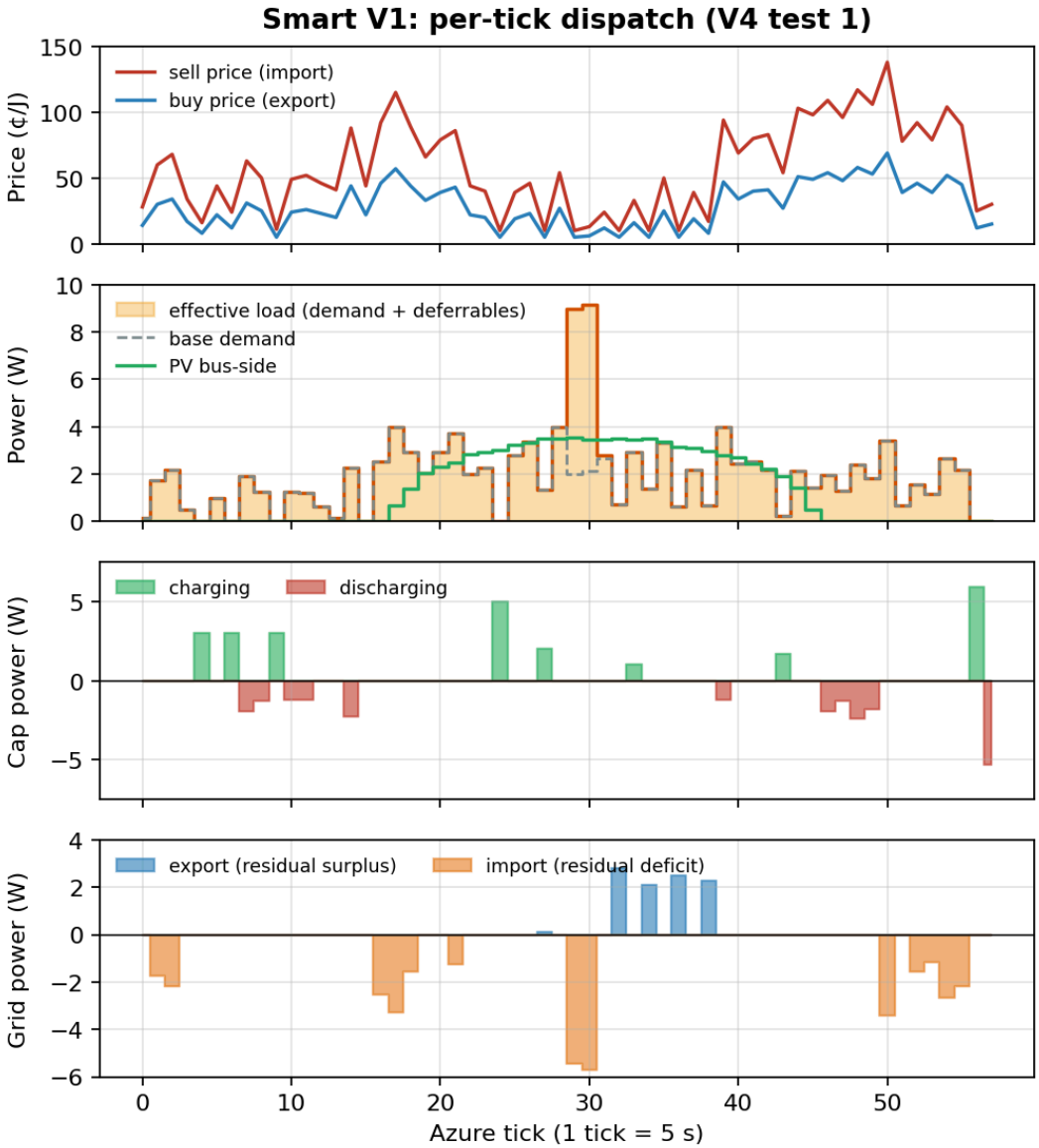


Figure 9.2: Smart V1: per-tick dispatch over the same Azure day (V4 test 1).

9.5 Evaluation

Each policy was run on the grid against the same fixed price, demand and irradiance profile (514 J base demand, 100.6 J of deferrables, 615 J LED target). Net cost integrates the grid module's `p_import` and the export module's `p_dissip` trapezoidally over every consecutive sample pair, multiplied by the sell or buy price in effect at the earlier timestamp; sample gaps longer than 30 s are skipped. The same integrator drives the `/algo/run/cost` endpoint the dashboard renders during a run, so the numbers in Table 9.1 are directly comparable to a live P&L card.

Run	Import (¢)	Export (¢)	Net (¢)	Saving vs naive
Naive, test 1	34,789	70	34,719	baseline
Naive, test 2	33,493	63	33,430	baseline
Smart, test 1	27,863	450	27,413	21.0%
Smart, test 2	27,447	384	27,064	19.0%

Table 9.1: V4 lab cost summary (cents). Smart V1 saves 20% on average.

Figure 9.3 attributes the saving to where each policy spent its import allowance. Smart V1 imports roughly 3.6 times as much energy in the cheapest band (below 30 ¢/J) and correspondingly less in the 30 to 100 ¢/J middle bands, which is where the naive policy is forced to draw whenever the bus sags. Total imported energy is almost the same across both policies (476 J for smart versus 463 J for naive, averaged across the two paired tests): the saving comes from when the energy is bought, not how much. The supercapacitor cycles about 2.2 times as hard under smart (120 J charged, 98 J discharged on average, versus 54 J and 49 J under naive) and the grid exports about five times as much (36 J versus 7 J on average), the export being PV surplus that exceeds the 9 W LED clamp on peak midday ticks while the supercapacitor is also fully charging. Most of the saving comes from price arbitrage on the supercapacitor: cheap-band imports store energy in the cap, which is discharged later to displace what would otherwise be 70 to 100 ¢/J imports.

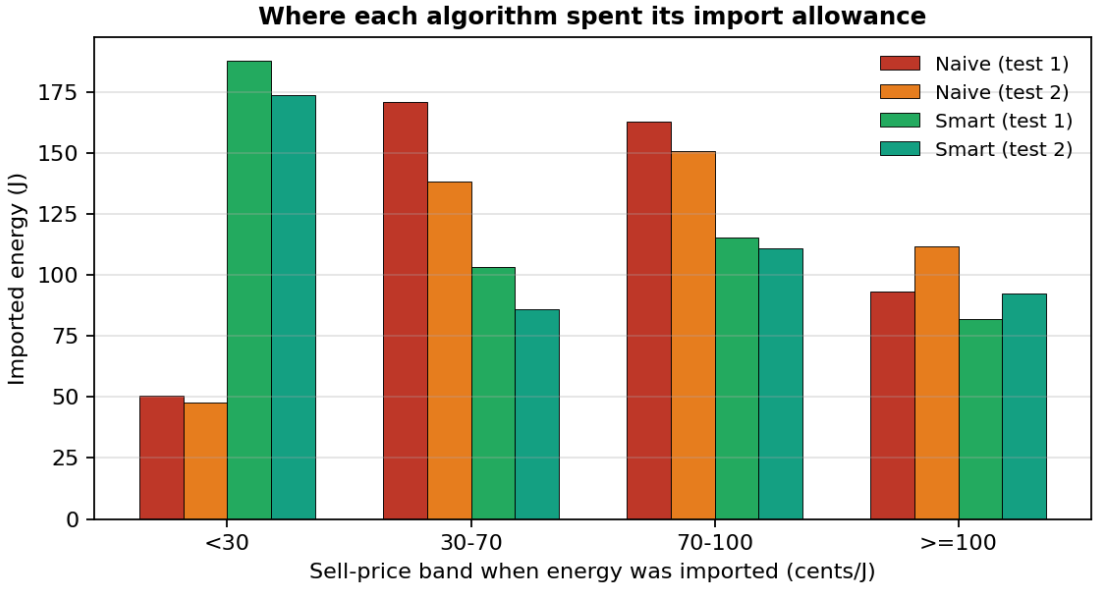


Figure 9.3: Imported energy by sell-price band: smart shifts imports into the cheap band.

9.6 Backtest on 1000 Simulated Days

The two lab runs sample a single scripted day, so both policies were also replayed offline over the 1000 randomly generated days in `datasets.json` to check that the saving holds across price profiles. The replay applies the same physical model as the firmware: PV follows the deterministic sun profile, the supercapacitor carries 90% SMPS efficiency in each direction with 0.25% leakage per tick, the LED load is clamped at 9W, droop closes the bus balance, and any deferrable energy left unserved at the end of a day is charged at that day's mean sell price.

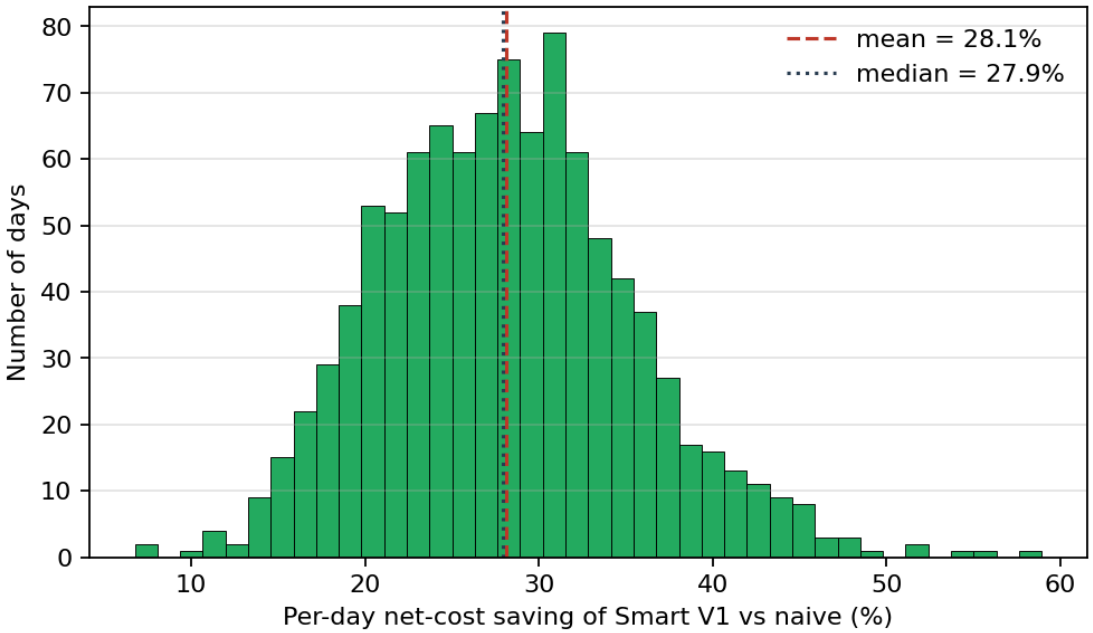


Figure 9.4: Distribution of daily saving of Smart V1 over naive across 1000 days.

Metric	Value
Mean daily saving	28.1%
Median daily saving	27.9%
Standard deviation	7.3%
Worst day / best day	6.8% / 58.9%
5th to 95th percentile	16.9% to 41.0%
Days where Smart V1 wins	1000 of 1000
Aggregate cost saving	27.9%

Table 9.2: Smart V1 vs naive backtest summary over 1000 simulated days.

Figure 9.4 shows the distribution is roughly bell shaped around a 28% saving and never crosses zero: Smart V1 beat the naive policy on every one of the 1000 days, with the worst day still 6.8% cheaper. The backtest mean sits above the 20% measured in the lab because the simulation has no command latency and no measurement noise, both of which blunt the timing of charge and discharge decisions on the real grid. The spread reflects how much each day's price profile offers: flat days leave little for the forecast to exploit, while a deep midday trough followed by a sharp evening peak rewards it most.

9.7 Auxiliary Power Overhead

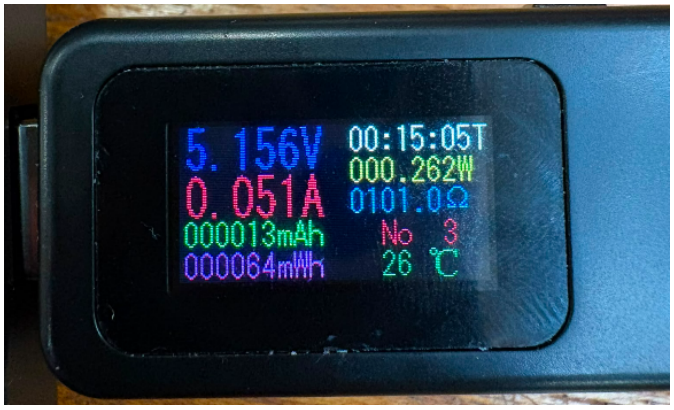


Figure 9.5: USB inline meter: a single Pico W draws 0.262 W on its 5 V rail.

The five Pico W controllers run from a separate USB powered hub, unmetered by the algorithm. A USB inline meter reads 0.262 W per module (Figure 9.5), so the aggregate fixed draw is 1.31 W or 393 J per 300 s Azure day. Total energy throughput across the grid (PV panel-side, supercapacitor charge and discharge, grid import and export, LED dissipation) averaged 1724 J per day, or 5.75 W. The controllers therefore account for $\frac{1.31}{1.31+5.75} \approx 19\%$ of total grid power. However, we note that this is a fixed cost: with a small-scale grid it is highly inefficient, but with a larger supercapacitor, higher demand, it can improve the efficiency.

10 Mechanical Design

10.1 3D-Modelled SMPS Enclosure

The four bidirectional SMPS boards sit in a 3D-printed chassis that protects the exposed power electronics from accidental shorts and keeps the assembly stable enough to move and demo without disturbing the wiring.

The v1 print (Figure 10.1, left) had solid side walls and a solid base. It was mechanically fine but trapped heat: with no intake from the sides or below, the lower boards had no convection path.

The v2 revision (Figure 10.1, right) added vertical slots in the side walls and vents in the base, so cool air enters from below and rises through to the open top. 3 mm standoff ribs on the inside walls hold each board off the wall so air flows between them.

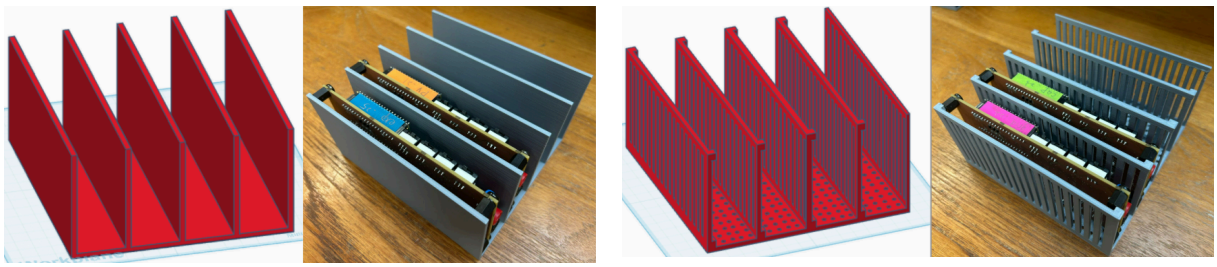


Figure 10.1: SMPS stand iteration: v1 (left) and v2 (right)

11 Extra Test & Requirement Verification

11.1 Communication & Integration Tests



Figure 11.1: Median Smart V1 latency breakdown across the 20-point dataset.

Metric	Value
Dataset size	20 traces
Mean end-to-end latency	1450 ms
Median end-to-end latency	1417 ms
Minimum end-to-end latency	597 ms
Maximum end-to-end latency	2797 ms
95th percentile end-to-end latency	2271 ms
Median Azure HTTP fetch latency	348 ms
Median state acquisition latency	471 ms
Median Smart V1 decision latency	2.1 ms
Median command dispatch RTT	307 ms
Median Pico control adoption latency	4.0 ms
Median data confirmation latency	942 ms
Median total latency including Azure fetch	1759 ms

Table 11.1: Summary statistics for Smart V1 end-to-end latency across the 20-point dataset.

A set of latency data was captured using an autonomous script during Smart V1 operation. After removing incomplete or invalid traces, a 20-point dataset remained for analysis. Each trace was associated with a unique trace identifier, allowing the same control event to be tracked from Azure tick acquisition, through Smart V1 decision generation and command dispatch, to Pico control-loop adoption and data confirmation.

Across the 20 traces, the median end-to-end latency from backend detection of a new Azure tick to data confirmation was 1417 ms, with a mean of 1450 ms. The minimum and maximum measured end-to-end latencies were 597 ms and 2797 ms respectively, while the 95th percentile was 2271 ms. Including the initial Azure HTTP fetch stage, the median total latency was 1759 ms.

The latency breakdown shows that Smart V1 computation and embedded control adoption were not significant contributors to the total delay. The median Smart V1 decision time was 2.1 ms, and the Pico control loop adopted the received command in 4.0 ms. In contrast, state

acquisition, command dispatch and data confirmation required median latencies of 471 ms, 307 ms and 942 ms respectively.

This indicates that the remaining delay in the system was dominated by communication and data handling rather than algorithm execution or low-level embedded control response. The test therefore confirms that the full Azure-to-backend-to-SMPS command path operated successfully under autonomous Smart V1 control, while also identifying the communication path as the main target for future optimisation.

11.2 Requirements Verification

Each of the brief's six requirements [10] is mapped below to the subsystem that realises it, the test that verifies it, and its current status. All six are implemented and verified by the tests in the cited sections.

Req	Requirement	Realised by	Verification	Status
1	Supply domestic LEDs per web demand	LED Load SMPS (Section 6)	Power-tracking test (Section 6.3)	Implemented; tracking within ±3% across channels
2	Extract PV energy with MPPT	PV SMPS (Section 3)	MPPT mode comparison; efficiency sweep	Implemented; P&O shipped as default with Fixed and LUT fallbacks
3	Store excess in supercapacitor, no batteries	Supercapacitor SMPS (Section 4)	Cycle test; round-trip efficiency	Implemented
4	Import and export to grid, metered	Import/Export SMPS (Section 5)	Import efficiency sweep; bus dead-band	Implemented
5	Minimise cost; schedule storage and deferrables; beat naive	Algorithm (Section 9)	Dry run versus naive baseline	Implemented; 20% average saving vs naive
6	UI for current and historic energy data	User Interface (Section 8)	Live dashboard; history plots	Implemented

Table 11.2: Brief requirements: verification method and current status.

References

[1] P. Clemow, "EE2 Power Lab SMPS Schematic 2026." [Online]. Available: https://github.com/edstott/EE2Project/blob/main/smart-grid/doc/EE2_Power_Lab_SMPS_Schematic_2026.pdf

[2] "ICElec Smart Grid Web Service." [Online]. Available: https://icelec50015.azurewebsites.net/

[3] R. W. Erickson and D. Maksimović, *Fundamentals of Power Electronics*, 3rd ed. Springer, 2020.

[4] T. Dragičević, X. Lu, J. C. Vasquez, and J. M. Guerrero, "DC Microgrids: Part II - A Review of Power Architectures, Applications, and Standardization Issues," *IEEE Transactions on Power Electronics*, vol. 31, no. 5, pp. 3528–3549, 2016.

[5] N. Femia, G. Petrone, G. Spagnuolo, and M. Vitelli, "Optimization of Perturb and Observe Maximum Power Point Tracking Method," *IEEE Transactions on Power Electronics*, vol. 20, no. 4, pp. 963–973, 2005.

[6] S. Dubey, J. N. Sarvaiya, and B. Seshadri, "Temperature Dependent Photovoltaic (PV) Efficiency and Its Effect on PV Production in the World – A Review," *Energy Procedia*, vol. 33, pp. 311–321, 2013, [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1876610213000829?via%3Dihub

[7] J. D. Navamani, M. L. Veena, L. Anbazhagan, and V. Krishnasamy, "Efficiency comparison of quadratic boost DC-DC converter in CCM and DCM," in *2015 International Conference on Circuit, Power and Computing Technologies (ICCPCT)*, IEEE, 2015. [Online]. Available: https://www.researchgate.net/figure/Efficiency-vs-duty-cycle-see-online-version-for-colours_fig5_283028397

[8] Cornell Dubilier, "Type DSM Supercapacitor, Datasheet Rev. 2." [Online]. Available: https://www.mouser.co.uk/datasheet/3/208/1/KNO_CD_DSM_Datasheet_R2.pdf

[9] L. Jamieson, T. Roy, and H. Wang, "Postulation of optimal charging protocols for minimal charge redistribution in supercapacitors based on the modelling of solid phase charge density," *Journal of Energy Storage*, vol. 33, p. 102075, 2021, [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S2352152X20319058?via%3Dihub

[10] E. Stott, "EE2 Project: Smart Grid – Requirements." [Online]. Available: https://github.com/edstott/EE2Project/blob/main/smart-grid/README.md

[11] Microchip Technology, "MCP3204/3208: 2.7V 4-Channel/8-Channel 12-Bit A/D Converters with SPI Serial Interface." [Online]. Available: https://ww1.microchip.com/downloads/aemDocuments/documents/APID/ProductDocuments/DataSheets/21298e.pdf

[12] P. Clemow, "LED Driver 2026." [Online]. Available: https://github.com/edstott/EE2Project/blob/main/smart-grid/doc/LED_Driver_2026.pdf

[13] Texas Instruments, "INA219 Zero-Drift, Bidirectional Current and Power Monitor with I2C Interface." [Online]. Available: https://www.ti.com/lit/ds/symlink/ina219.pdf

[14] Raspberry Pi Ltd, "RP2040 Datasheet." [Online]. Available: https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf